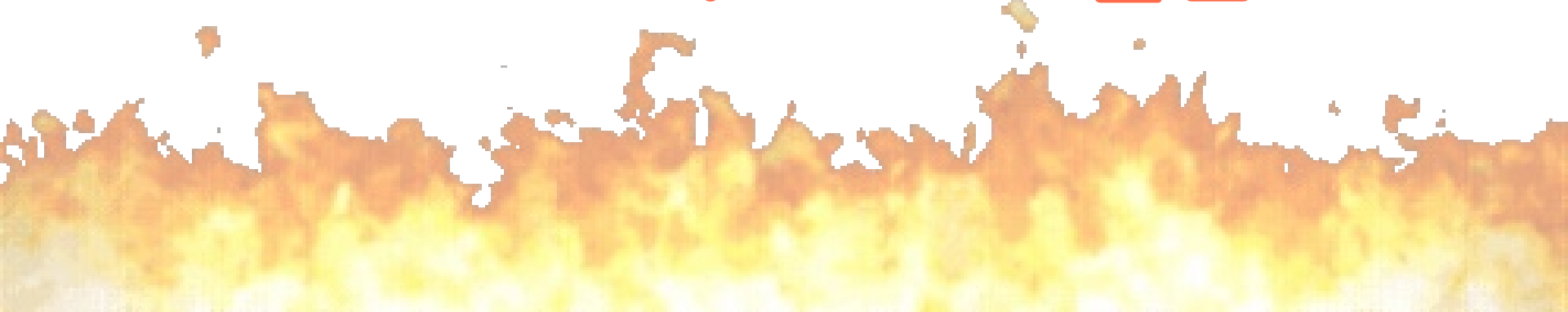


뉴런데브클래스

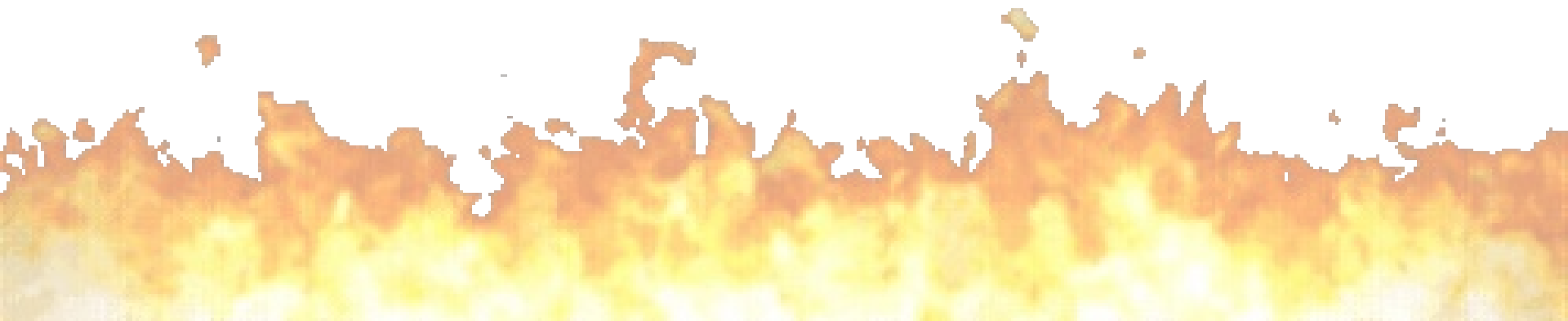
현대 AI 의 원리와 구조

#4-2 벡터디비 실습



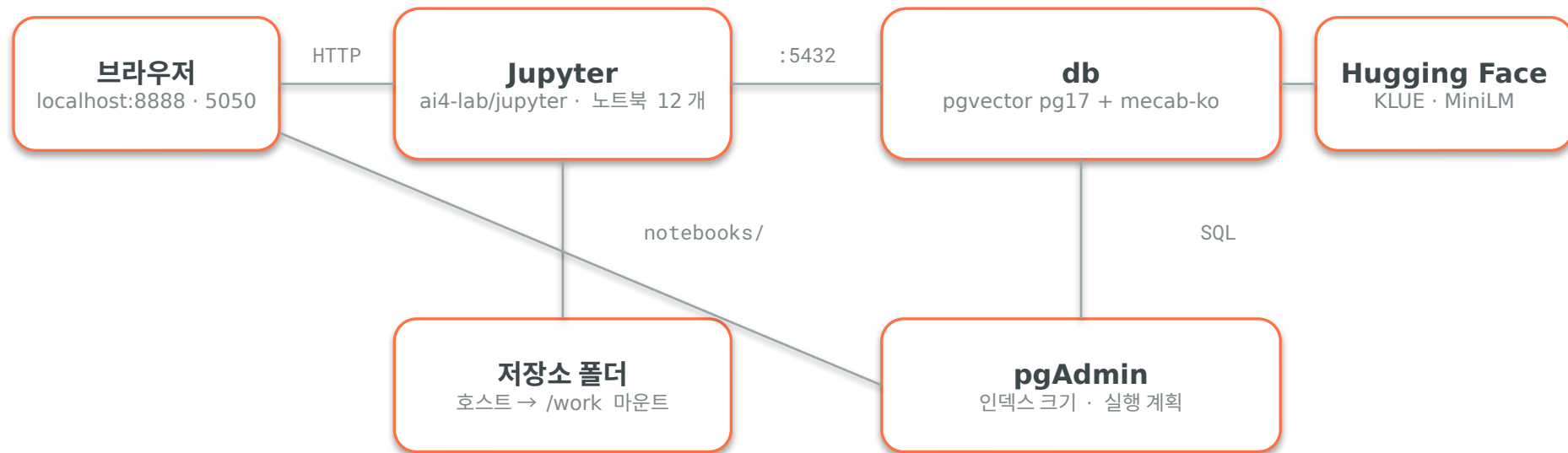
목차

- 1 실습 환경
- 2 NSW · HNSW 직접 구현
- 3 IVF · PQ 직접 구현
- 4 pgvector 인덱스 측정
- 5 필터와 벡터 검색
- 6 점수 퓨전과 분포 진단



1. 강의의 인덱스 원리를 numpy 로 처음부터 만들어 본다
 - NSW 그리디 → HNSW 계층 → IVF(k-means) → PQ(ADC)
 - 그래프로 탐색 경로 · 보로노이 경계 · 거리표를 눈으로 확인
2. 같은 현상을 pgvector 로 재 본다
 - 합성 벡터 10 만 개 · 정확한 정답지 → recall@10 과 지연시간
 - HNSW · IVFFlat · 양자화 · 필터 전략을 숫자로 비교
3. 정답이 있는 한국어 데이터로 검색 품질을 진단한다
 - KLUE-MRC 질문 500 개 (4-1 과 같은 데이터 · BM25)
 - RRF vs Min-Max · 무릎점 · 분리도 · 필터 적합성 · 허브
4. 모든 실습은 Jupyter 노트북
 - 4-1 의 docker-compose 하나에 Jupyter 를 더했다
 - 실행하면 그래프 40 여 개가 그려진다

컨테이너 3 개 + 저장소 폴더 + 외부 자원



볼륨 : pgdata(DB · 4-1 과 공유) · pgadmin · hfcache(데이터셋 · 임베딩 모델 , loader 와 공유)
Jupyter 는 저장소 루트를 /work 로 연다 → 4 강실습 /loader · sql 을 import 해서 재사용

90 분 진행표

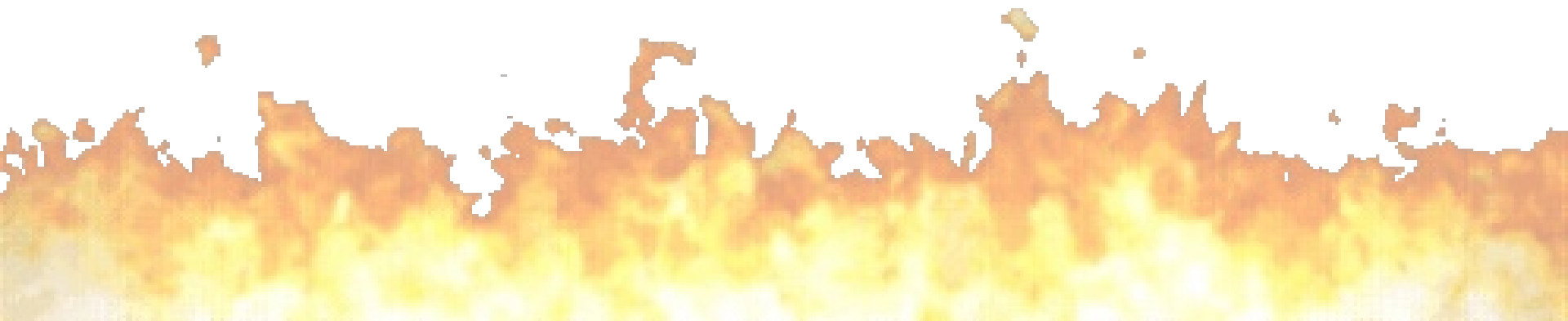


기계가 계산하는 시간은 5 분 남짓

시간	단계	노트북	계산 시간 (12 코어)
00-10	① 환경	compose up → Jupyter → 00	이미지 빌드 1 분 30 초
10-25	② NSW · HNSW	10 · 11	7 초 · 43 초
25-40	③ IVF · PQ	12 · 13	12 초 · 37 초
40-60	④ pgvector	20 · 21 · 22 · 23	8 초 · 70 초 · 41 초 · 22 초
60-70	⑤ 필터	24	29 초
70-90	⑥ 퓨전 · 분포	30 · 31	20 초 · 11 초

00 번이 시작하는 KLUE 준비 (청크 27,943 개 임베딩) 는 12 코어에서 약 15 분 — ② ~ ⑤ 동안 뒤에서 끝난다
4 코어 노트북은 2~3 배 → 00 번을 가장 먼저 . ⑥ 전에 '✓ KLUE 준비 완료' 확인

실습 환경



4-1 컴포즈에 jupyter 서비스와 빌드 설정을 더했다

```
db:
  command: >
    postgres -c shared_buffers=512MB
    -c maintenance_work_mem=1GB
    -c max_parallel_maintenance_workers=4 -c jit=off
  shm_size: 1g          # 병렬 빌드용
jupyter:
  build: ./jupyter
  ports: ["${JUPYTER_PORT:-8888}:8888"]
  volumes:
    - ../work           # 저장소 루트
    - hfcache:/cache    # loader 와 공유
```

1. 실행
 - cd -ai-/4 강실습
 - docker compose up -d --build
2. 4-1 을 했다면
 - jupyter 이미지만 새로 빌드
 - db 는 설정이 바뀌어 재생성 , 데이터는 그대로
- 3.shm_size 가 없으면
 - 병렬 HNSW 빌드가 공유 메모리 부족으로 실패 (도커 기본 64MB)

python:3.12-slim + 과학 계산 + 한글 글꼴

```
FROM python:3.12-slim
RUN apt-get install -y fonts-nanum          # 그래프 한글
RUN pip install jupyterlab numpy pandas matplotlib \
    psycopg[binary] pgvector fastembed pyarrow
ENV PYTHONPATH=/work/4 강실습 /loader      # 4-1 load.py 재사용
CMD jupyter lab --IdentityProvider.token= \
    --ServerApp.root_dir=/work \
    --LabApp.default_url=/lab/tree/4-2 강실습 /notebooks
```

1. <http://localhost:8888> → 4-2 강실습 /notebooks 가 바로 열린다 (로그인 없음)
- 로컬 실습 전용 설정 — 외부에 노출하지 말 것

2. 셀 실행 Shift+Enter · 전체 실행 Run → Run All Cells

3 부 데이터를 백그라운드로 먼저 시작한다

```
import subprocess
subprocess.Popen("python prepare_klue.py > prepare.log 2>&1", shell=True)
# 1) 텍스트 2) BM25 역색인 3) 지문 임베딩 4) 청크 임베딩 - 이미 있으면 건너뛴
```

확인 항목	값
PostgreSQL	17.11
pgvector	0.8.6 (iterative scan 은 0.8 부터)
maintenance_work_mem	1GB
병렬 빌드 워커	4

청크 = 문장 경계로 250 자 , '제목 + 청크' 를 임베딩 (Contextual Retrieval 을 가장 단순하게 흉내)
지문 5,309 개 → 청크 27,943 개 (평균 5.3 개). 4-1 을 마쳤다면 청크 단계만 한다

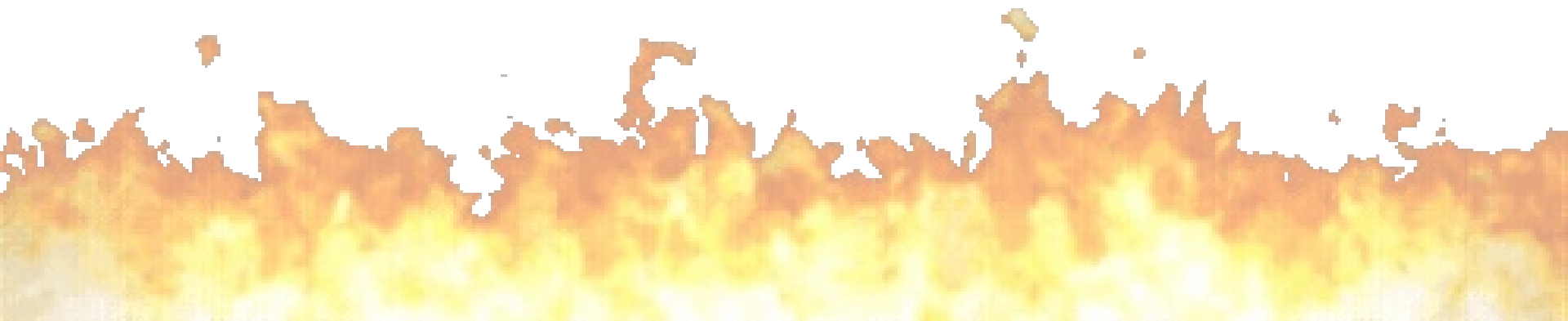
모든 노트북이 쓰는 공용 도구

```
def connect():
    conn = psycopg2.connect(DSN, autocommit=True)
    conn.prepare_threshold = None          # ← 이 줄이 없으면 측정이 틀어진다
    register_vector(conn)                 # numpy 배열 ↔ vector

def pg_bench(conn, table, ds, k=10):      # 질의 200 개 → recall@k · 지연시간
def brute_knn(X, Q, k):                   # 정확한 k-NN = 모든 실험의 정답지
```

psycopg2 는 같은 SQL 을 여러 번 실행하면 서버에 prepare 해 두고 실행 계획을 재사용한다
SET hnswe.ef_search 를 바꿔도 옛 계획이 쓰인다 — 실제로 모든 설정에서 recall 이 같게 나왔다

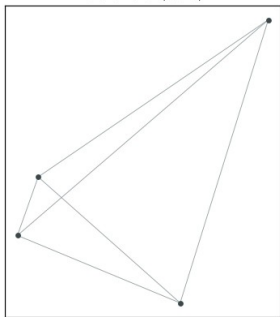
NSW · I·NSW 직접 구현



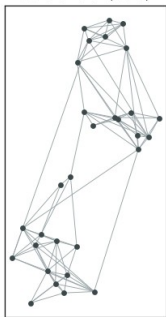
점을 하나씩 넣으며 가까운 M 개와 친구 맺기

```
for i in range(1, len(X)):  
    for j in np.argsort(dist(X[:i], X[i]))[:M]: # 이미 있는 점 중 가까운 M 개  
        nbrs[i].append(j); nbrs[j].append(i) # 양방향  
        if len(nbrs[j]) > 2 * M: # 넘치면 가장 먼 친구를 끊는다  
            nbrs[j].remove(max(nbrs[j], key=...))
```

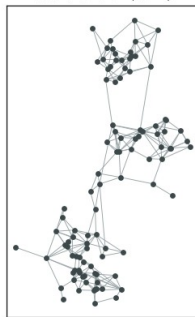
4개 추가 후 (M=4)



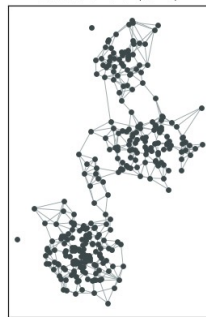
30개 추가 후 (M=4)



100개 추가 후 (M=4)



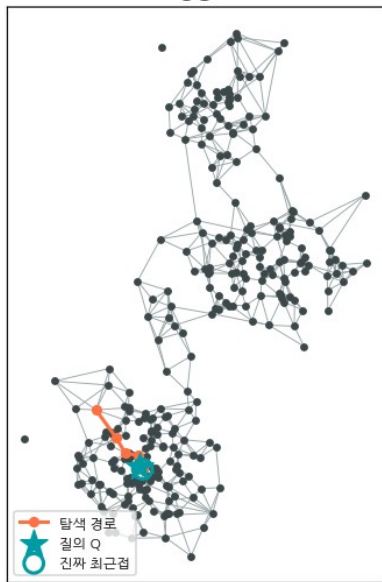
300개 추가 후 (M=4)



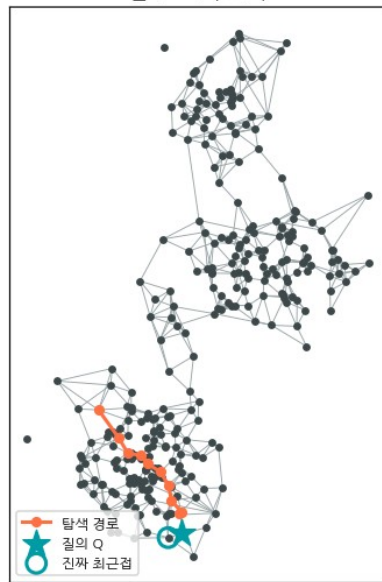
먼저 들어온 점은 멀어도 연결된다 → 지름길 . 친구까지 평균 거리 : 처음 30 개 0.57 · 마지막 100 개 0.38

더 가까운 친구가 없으면 멈춘다

성공



실패: 지역 최적



1. 질의 500 개 중 최근접을 맞힌 비율 35%
- 평균 거리 계산 29 회 (전수 300 회)
2. 오른쪽 : 친구들이 모두 멀어 멈췄다
- 그 너머에 진짜 최근접 = 지역 최적
3. 강의 슬라이드 6 의 '운 나쁘면 진짜 최근접을 놓친다'

한 점이 아니라 후보 ef 개를 들고 다닌다

M	도달 가능	ef = 1	ef = 4	ef = 16
2	49%	0.06 / 9 회	0.07 / 11 회	0.43 / 39 회
4	99%	0.35 / 20 회	0.78 / 43 회	0.99 / 64 회
6	100%	0.93 / 46 회	1.00 / 53 회	1.00 / 66 회

1. 칸 = recall@1 / 질의당 거리 계산 (전수 300 회)

- ef ↑ → recall ↑ · 비용 ↑ — HNSW 의 ef_search 가 이것

2. M=2 는 군집끼리 이어지지 않아 섬이 생긴다

- 진입점에서 절반에만 도달 → ef 를 키워도 섬 밖은 못 찾는다

$\text{층} = \text{floor}(-\ln U \times mL), mL = 1/\ln M$

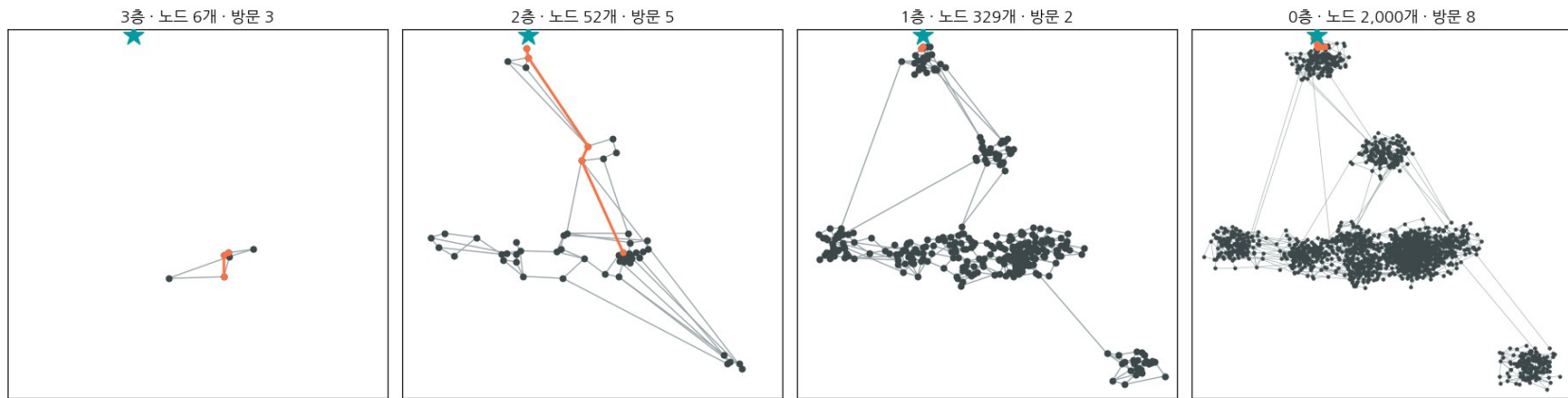
```
lvl = int(-np.log(rng.random()) * mL)    # P( 층 ≥ l ) = M(-l)
```

층	0	1	2	3	4	5
올라간 노드 (100 만 , M=16)	1,000,000	62,397	3,937	260	15	1
기대값 $N \cdot M^{(-\text{층})}$	1,000,000	62,500	3,906	244	15	1

한 층 올라갈 때마다 노드가 $1/M$ 로 준다 — KTX 역처럼 위로 갈수록 드물다
0 층은 이웃 $2M$ 개 , 위층은 M 개 (논문 권장)

층을 내려오며 좁힌다

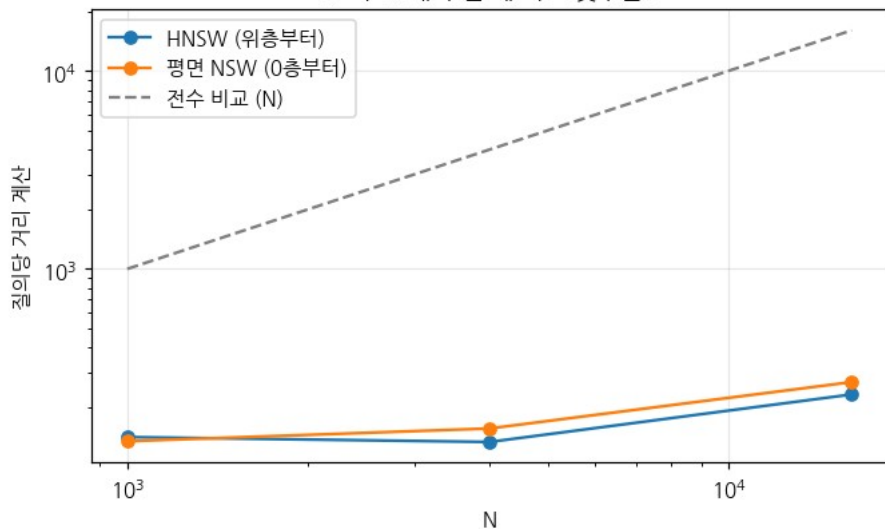
위층에서 멀리 건너가고, 0 층에서 정밀하게



1. 주황 = 방문한 노드 . 위층은 노드가 몇 개뿐이라 몇 걸음에 질의 (★) 쪽으로 건너간다
2. 아래층은 위층이 찾아 준 출발점에서 시작 → 헛걸음이 적다

같은 그래프를 위층부터 vs 0 층부터

N 이 16배가 될 때 비교 횟수는?



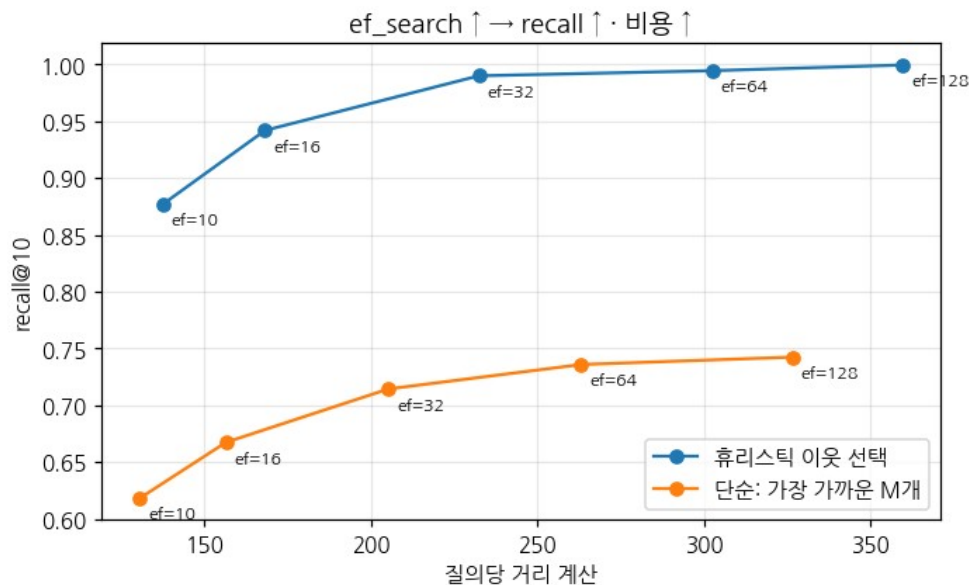
N	위층부터 recall / 비교	0 층부터 recall / 비교
1,000	1.000 / 142	1.000 / 135
4,000	0.999 / 134	0.990 / 157
16,000	0.990 / 232	0.810 / 268

1.N 이 16 배 → 비교는 142 → 232 회 (전수는 16 배)

2.N 이 커지면 0 층부터는 군집 사이를 못 건넌다

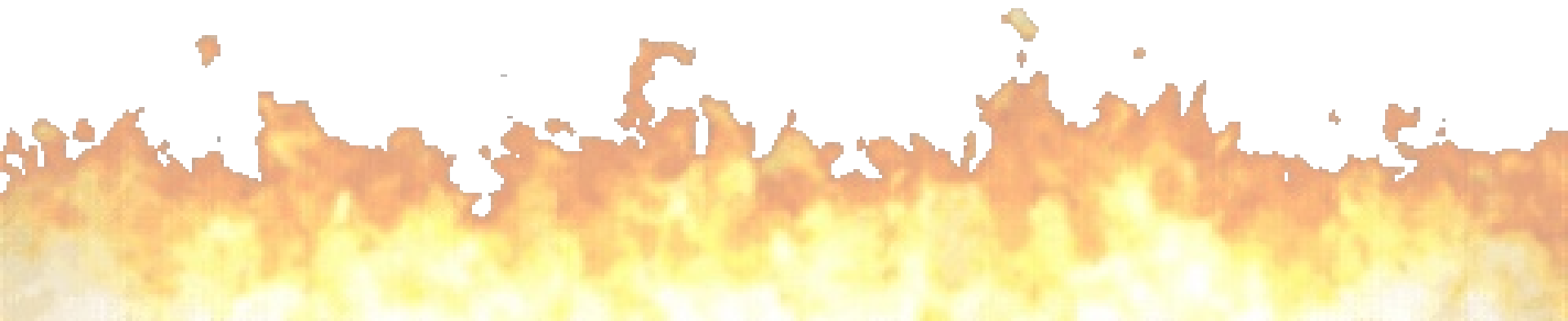
강의의 '가장 가까운 M 개' 와 실제 HNSW 의 차이

```
for d, c in candidates:                                # base 에 가까운 순
    if len(out) == M: break
    if not out or d < dist(X[c], X[out]).min():         # 이미 고른 이웃보다 base 에 가까울 때만
        out.append(c)
```



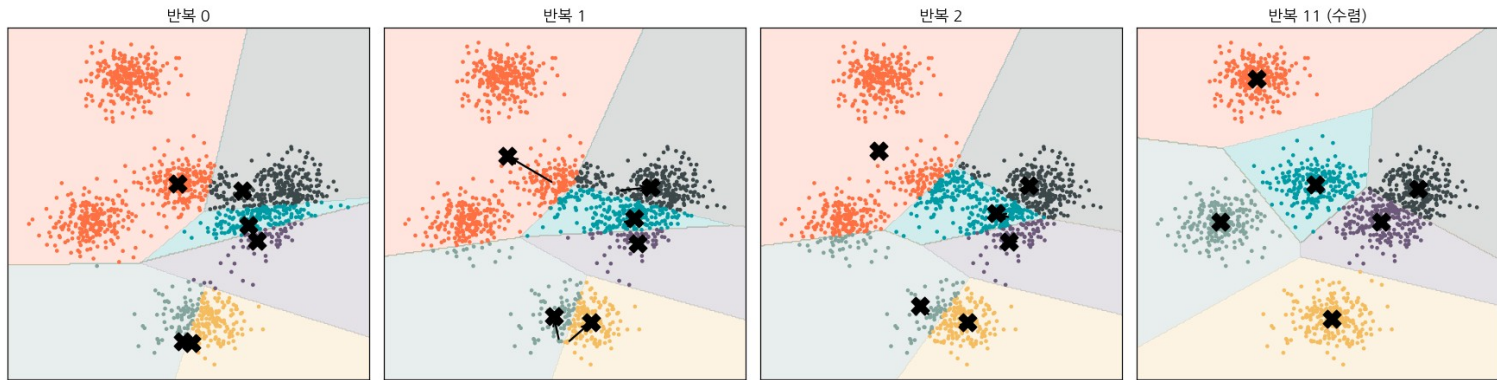
- 16 차원 군집 1 만 6 천 개 , ef=32
- 휴리스틱 0.990 · 가장 가까운 M 개 0.715
2. 가까운 것만 고르면 이웃이 한 군집에 몰린다
- 다른 군집으로 가는 다리가 약해진다

IVF · PQ 직접 구현



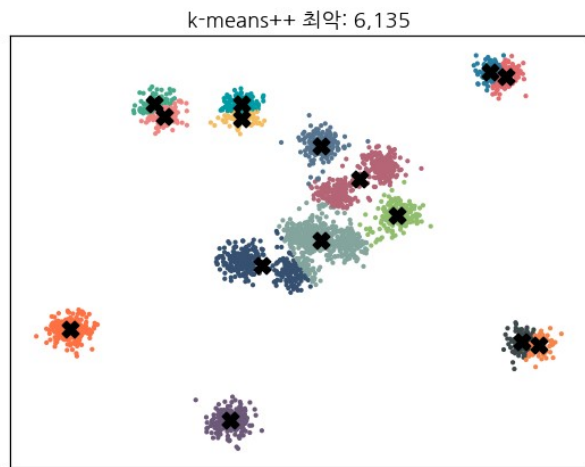
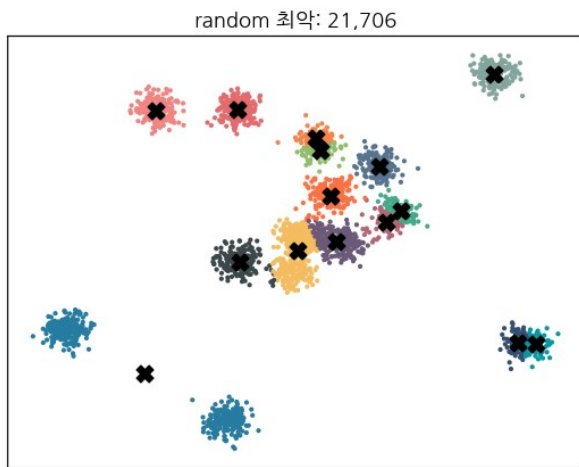
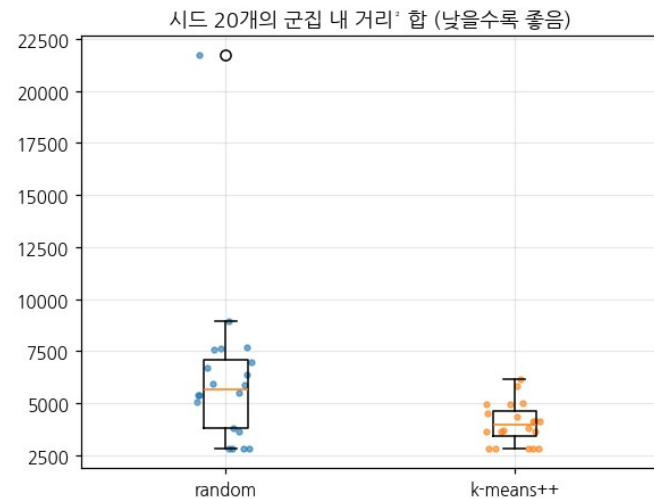
할당 → 갱신 → 수렴

```
lab = sqdist(X, C).argmin(1) # ② 가장 가까운 중심에 할당  
C = np.array([X[lab == j].mean(0) for j in range(k)]) # ③ 멤버 평균으로 갱신
```



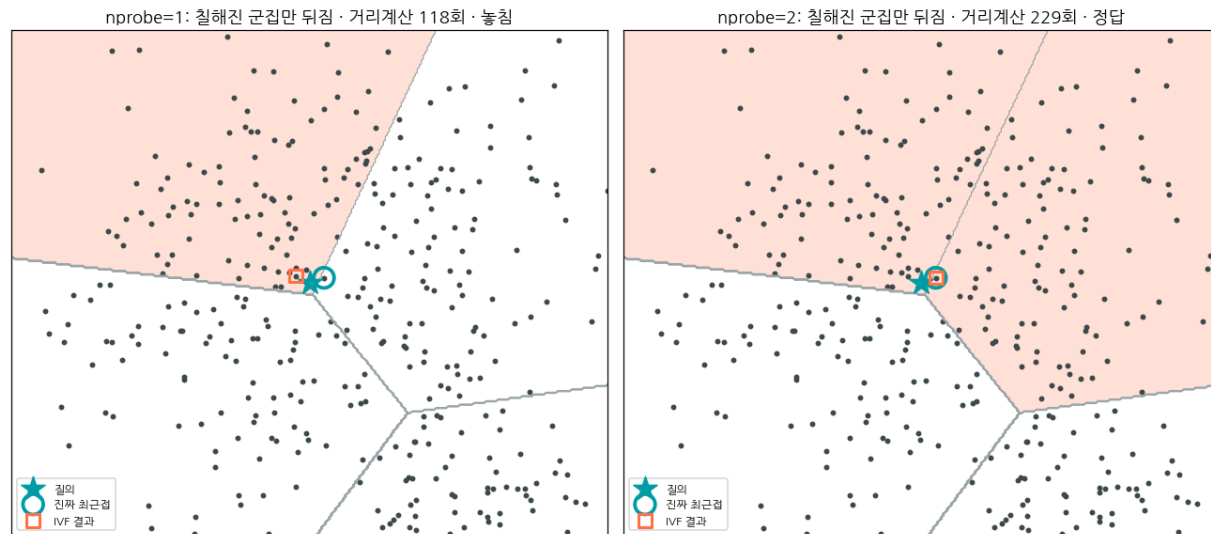
배경색 = 이 위치의 질의가 가는 군집 (보로노이 영역) · 화살표 = 중심이 평균으로 이동

무작위 시작 vs k-means++



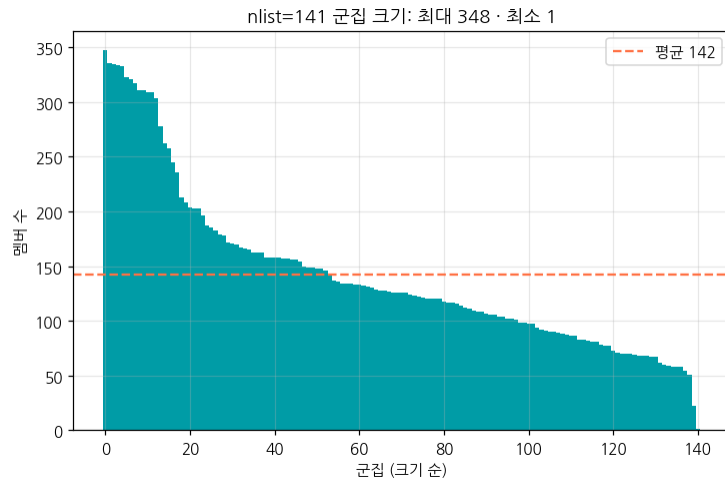
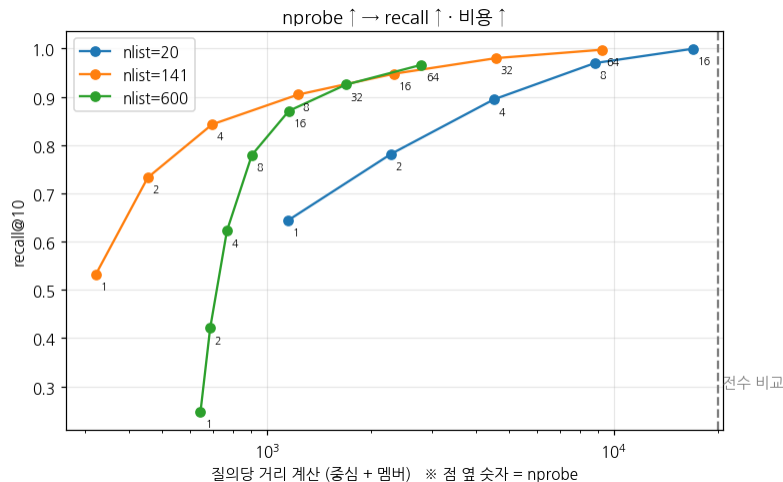
시드 20 개 : 무작위 시작은 한 군집에 중심 두 개가 몰리는 실패가 생긴다 (최악 21,706)
k-means++ 는 먼 점일수록 중심으로 잘 뽑혀 실패가 드물다 (최악 6,135)

가까운 nprobe 개 군집만 뒤진다



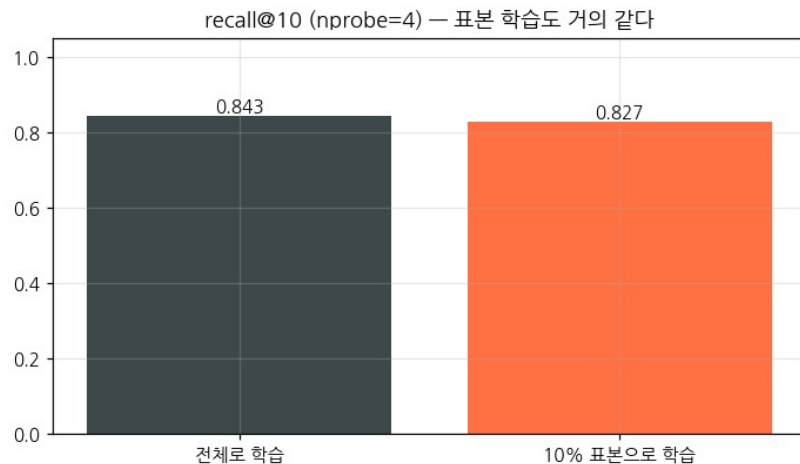
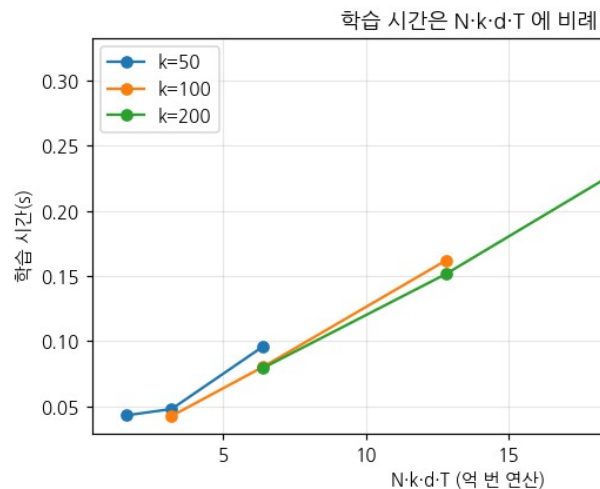
질의를 A 영역, 진짜 최근접은 경계 너머 B 군집 → nprobe=1 은 놓치고 (118 회) nprobe=2 는 찾는다 (229 회)

32 차원 2 만 개 , 군집끼리 겹치게



1. $nlist \approx \sqrt{N(141)}$ 곡선이 같은 비용에서 recall 이 가장 높다
- 20 은 군집이 커서 비싸고 , 600 은 경계가 많아 recall ↓
2. 군집 크기 1 ~ 348 — 큰 리스트를 뒤지는 질의만 느리다

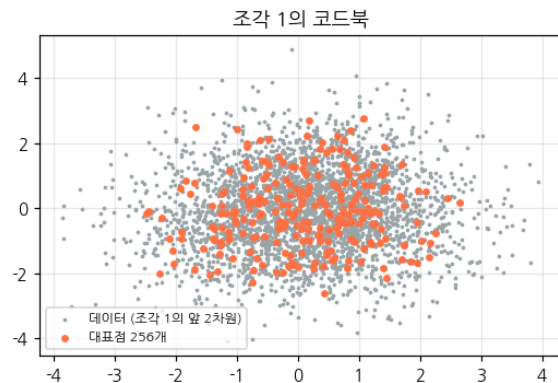
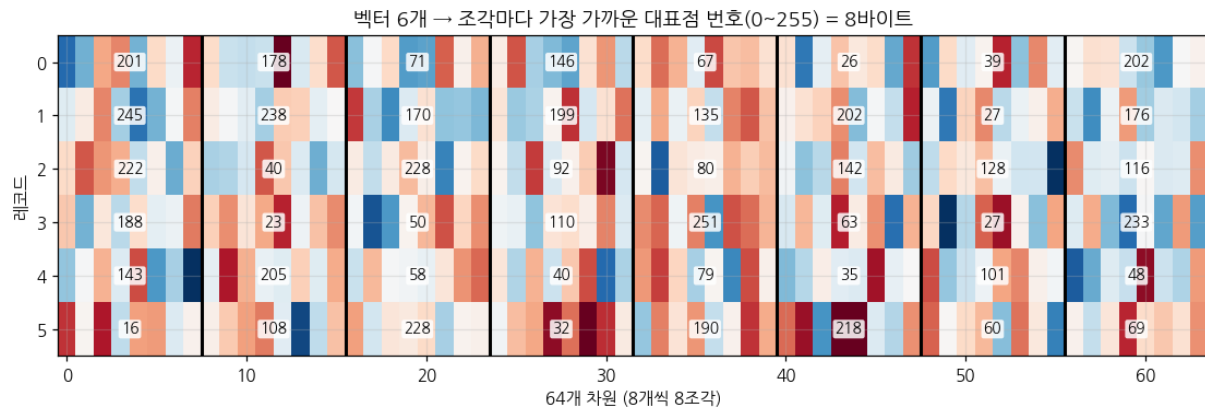
표본으로 중심만 학습해도 된다



반복 $T=10$ 고정, 3 회 중 최솟값 : 학습 시간이 $N \cdot k \cdot d \cdot T$ 에 비례
10% 표본으로 중심을 학습하고 전체는 배정만 해도 recall 0.843 $\rightarrow$ 0.827

PQ 코드 - 8 바이트

64 차원 → 8 조각 × 256 대표점



벡터 하나 = 조각마다 대표점 번호 (0~255) 8 개 = 8 바이트 (float32 256B 의 1/32)
강의 슬라이드 14 의 '8 차원 압축' 이 아니라 8 바이트 코드 — 좌표가 아니라 번호다

질의를 압축하지 않고 8×256 거리표를 만든다

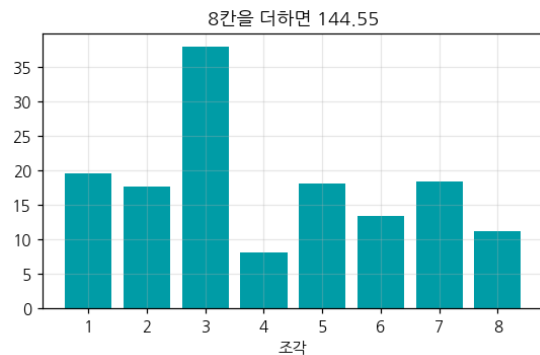
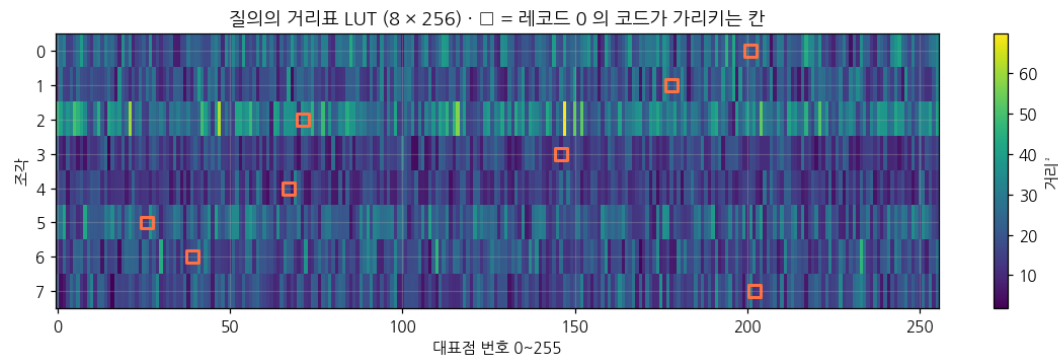


표 조회 합

$$\|q - \hat{x}\|^2 = 144.5538$$

복원 벡터로 직접 계산

$$\|q - \hat{x}\|^2 = 144.5538$$

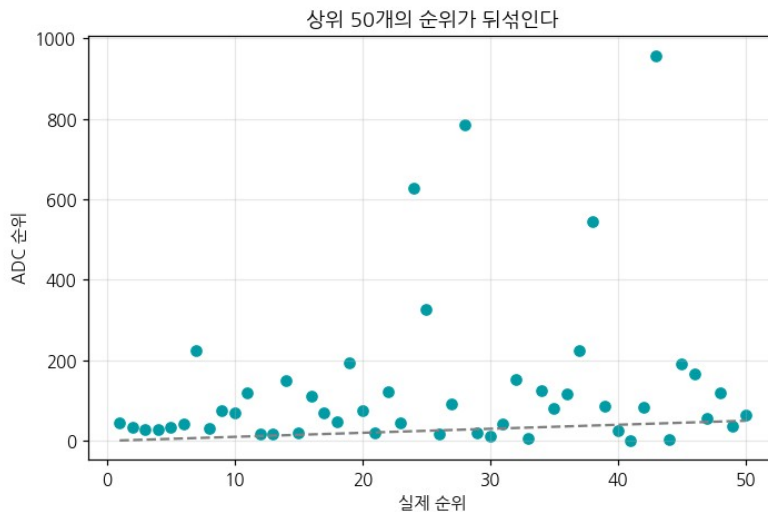
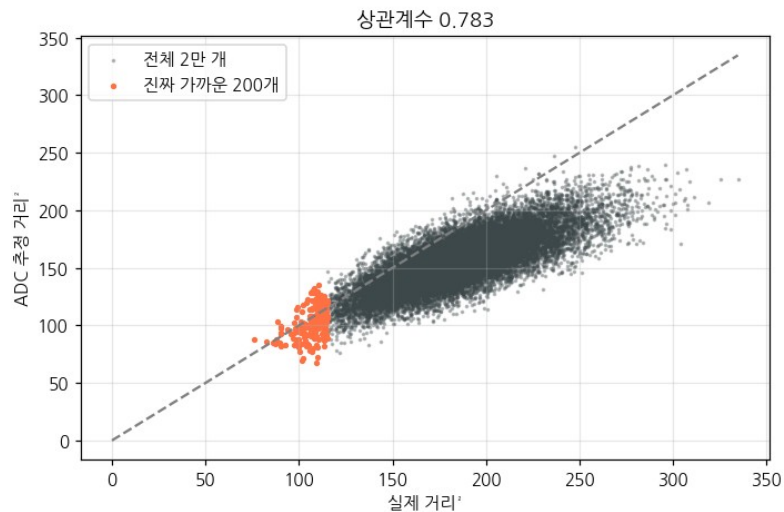
← 똑같다 (거리²는 조각별 합)

원본 벡터와의 실제 거리

$$\|q - x\|^2 = 187.2108$$

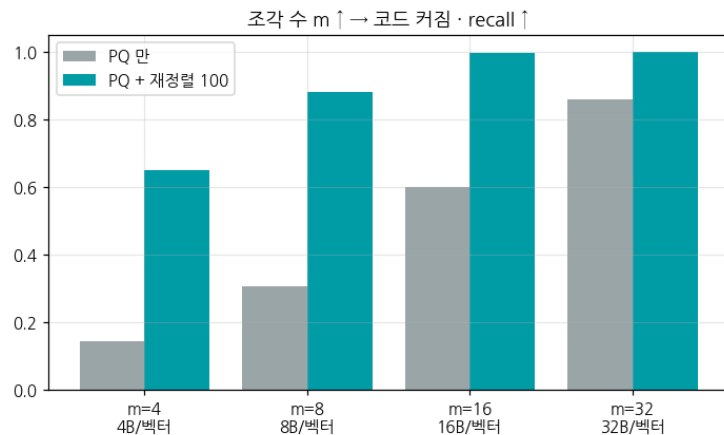
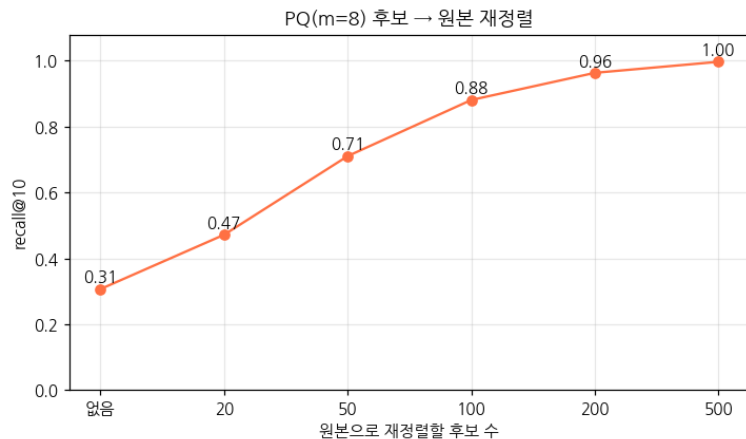
← 이 차이가 압축 손실

대략은 맞고, 가까운 쪽 순위가 뒤섞인다



손실 압축이라 가까운 후보들끼리의 순서를 잘 못 가린다 → 후보를 넉넉히 뽑아 원본으로 재정렬

IVF-PQ 가 표준인 이유

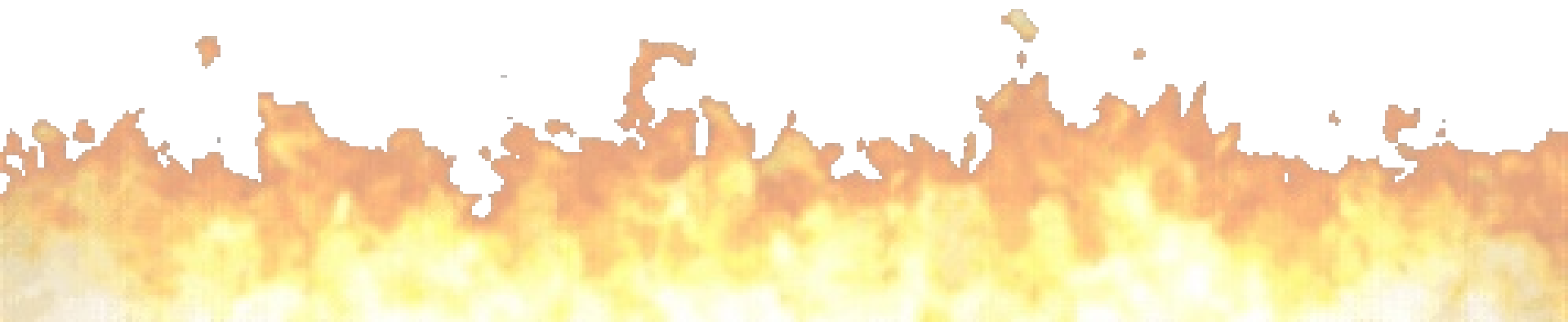


1. PQ 만 0.31 → 후보 100 개 재정렬 0.88 · 500 개 1.00

-m=16(16B) + 재정렬 100 이면 1.00

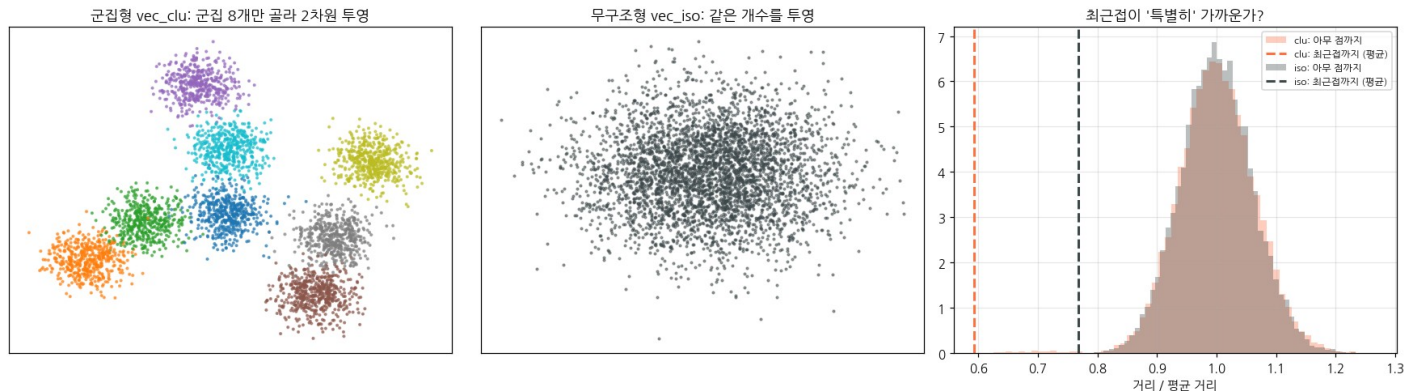
2. 잔차 (벡터 - 군집 중심) 를 압축하면 같은 8 바이트로 오차 29.6 → 18.8

pgvector 인덱스 측정



군집형 · 무구조형, 각 10 만 × 128 차원

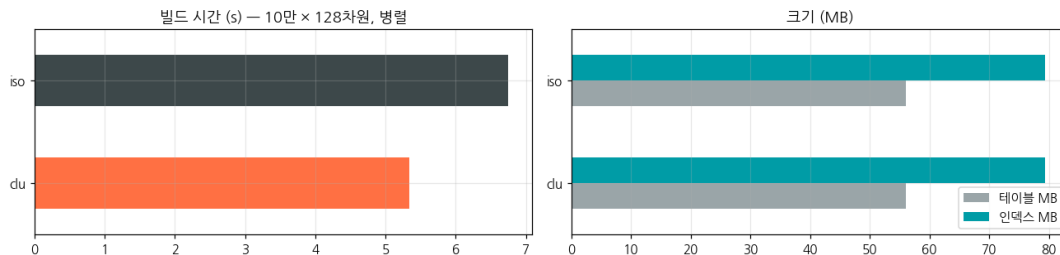
```
with cur.copy("COPY vec_clu (id, cluster, tag, emb) FROM STDIN WITH (FORMAT BINARY)") as cp:
    cp.set_types(["int4", "int4", "int4", "vector"])          # 10 만 행 1 초
idx = brute_knn(X, Q, 100)                                   # 질의 200 개의 정답 top-100 → vec_gt
```



최근점 / 평균 거리 : 군집형 0.59 · 무구조형 0.77 — 고차원에서는 모든 점이 비슷하게 멀다 (거리 집중)

인덱스가 테이블보다 크다

```
CREATE INDEX vec_clu_hnsw ON vec_clu USING hnsw (emb vector_l2_ops)
WITH (m = 16, ef_construction = 64); -- 5.0 초 (병렬)
```



	빌드	인덱스	테이블
균집형	5.0 초	79.4MB	59.2MB
무구조형	7.0 초	79.4MB	56.0MB

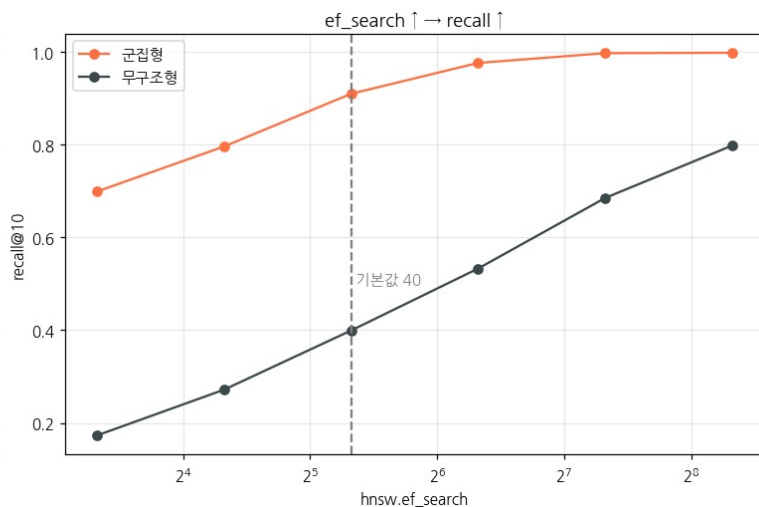
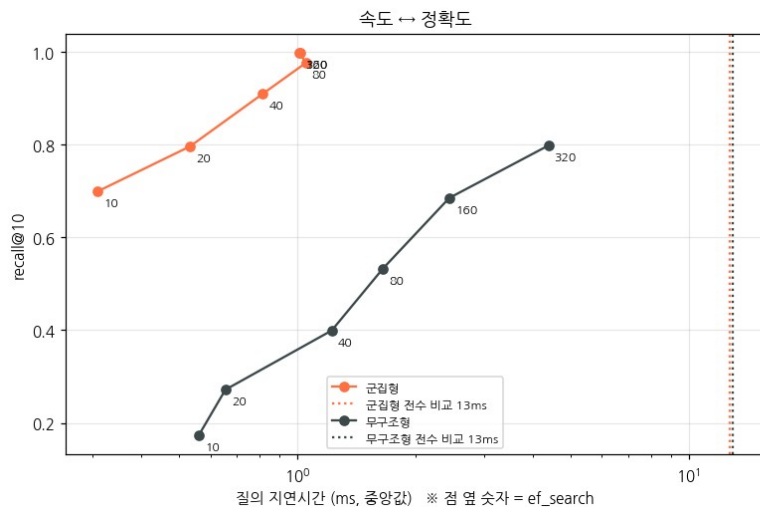
metric-agnostic 이지만 인덱스는 척도 하나에 묶인다

```
EXPLAIN SELECT id FROM vec_clu ORDER BY emb <-> $q LIMIT 10;  
Index Scan using vec_clu_hnsw on vec_clu          -- 1.2ms
```

```
EXPLAIN SELECT id FROM vec_clu ORDER BY emb <=> $q LIMIT 10;  
Parallel Seq Scan on vec_clu                      -- 14ms  
-- vector_l2_ops 인덱스는 <=>( 코사인 ) 에 쓰이지 않는다
```

HNSW 알고리즘은 어떤 척도로도 만들 수 있지만, 만든 인덱스는 빌드 때 고른 척도에만 쓰인다
임베딩이 정규화돼 있으면 코사인 · 내적 · L2 순위가 같으니 질의와 인덱스의 연산자만 맞추면 된다

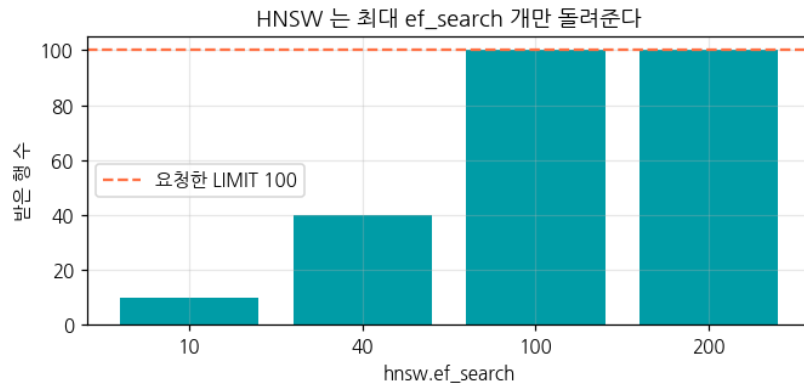
SET hnswef_search = 10 ... 320



기본값 40 : 군집형 recall 0.916 · 무구조형 0.406 — 같은 설정이라도 데이터에 따라 크게 다르다
전수 비교 (점선, 약 13ms) 보다 10 배 이상 빠르다. 1ms 아래는 잡음이 섞이므로 경향만 본다

함정 : LIMIT > ef_search

오류도 경고도 없이 적게 온다

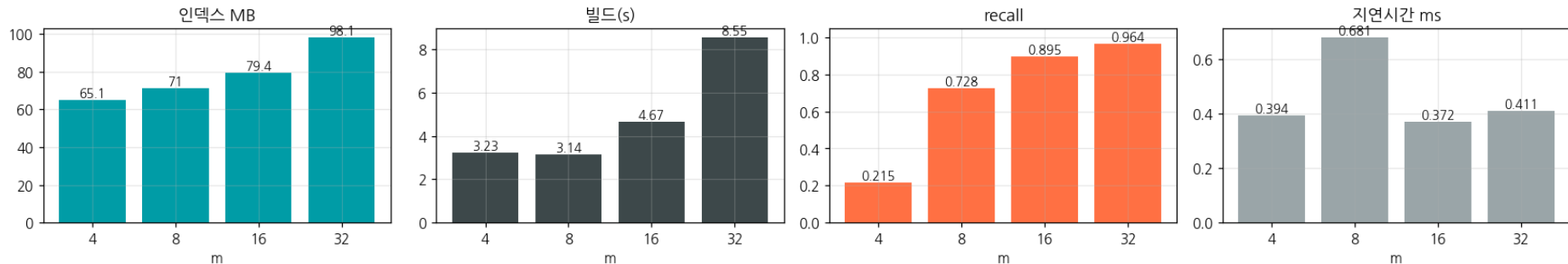


```
SET hnsw.ef_search = 40;
SELECT id FROM vec_clu ORDER BY emb <-> $q LIMIT 100;    -- 40 행
```

재정렬용 후보를 넉넉히 받으려면 $ef_search \geq LIMIT$ (또는 iterative scan)

이웃 목록이 커지면 인덱스 · 빌드 · recall 이 함께 는다

$m \uparrow \rightarrow$ 이웃 목록이 커져 인덱스·빌드 $\uparrow$, recall $\uparrow$ (ef_search=40)



1. $m \ 4 \cdot 8 \cdot 16 \cdot 32 \rightarrow$ recall $0.28 \cdot 0.71 \cdot 0.92 \cdot 0.97$ (ef_search 40)

- 인덱스 65 $\rightarrow$ 98MB — 대부분은 벡터 복사본이라 두 배까지 안 커진다

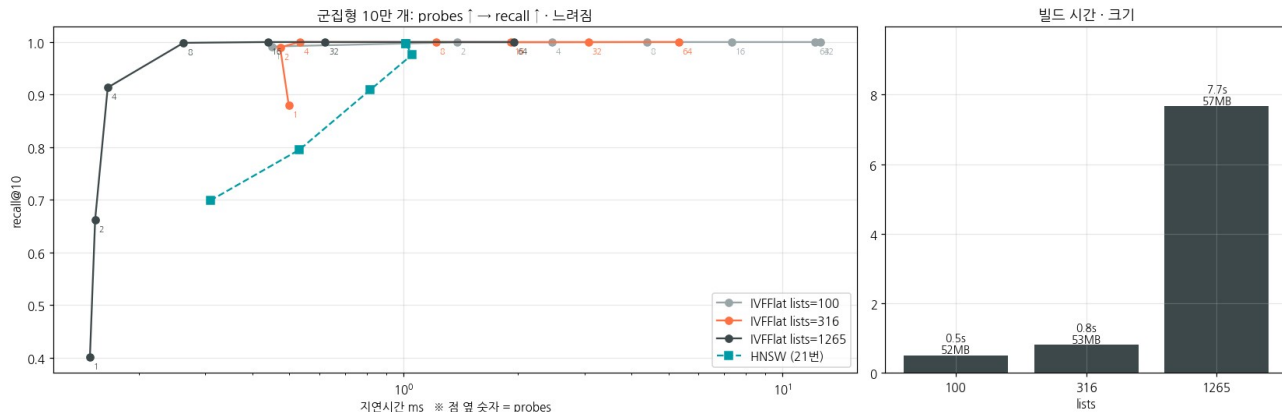
2. INSERT 는 즉시 검색된다 (2,000 건 건당 수 ms). 삭제는 표시 후 VACUUM 이 이웃을 다시 잇는다

IVFFlat : lists · probes



k-means 중심 + 역색인

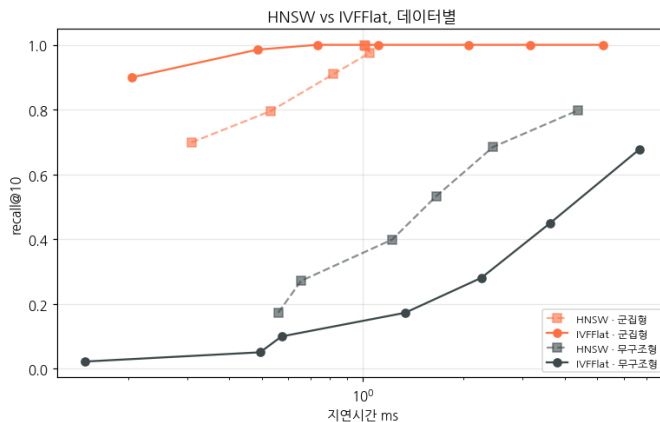
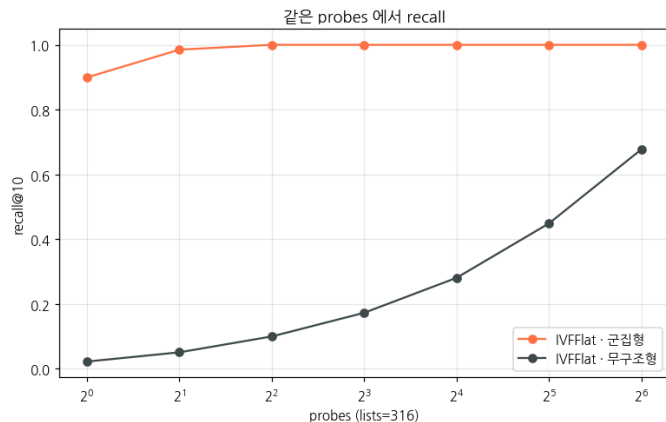
```
CREATE INDEX vec_clu_ivf ON vec_clu USING ivfflat (emb vector_l2_ops) WITH (lists = 316);  
SET ivfflat.probes = 4;          -- 기본값 1
```



probes=1(기본값) recall 0.86 — 기본값 그대로 쓰지 말 것 . 빌드 lists 100 · 316 → 0.5 · 0.8 초 , 1265 → 8 초

승자는 데이터가 정한다

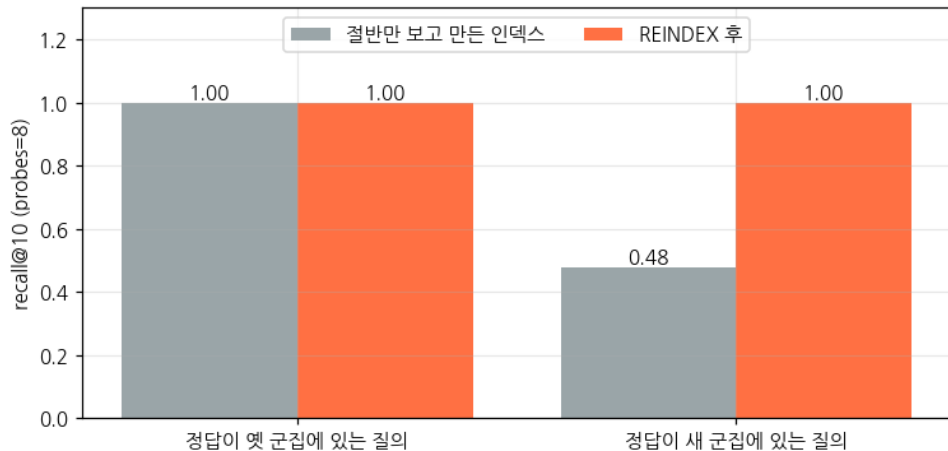
IVFFlat vs HNSW, 군집형 vs 무구조형



데이터	HNSW ef_search 40	IVFFlat probes 4	IVFFlat probes 64
군집형 (중심 200 개)	0.916	1.000	1.000
무구조형 (등방 가우시안)	0.406	0.109	0.668

IVFFlat 중심은 INSERT 때 다시 학습하지 않는다

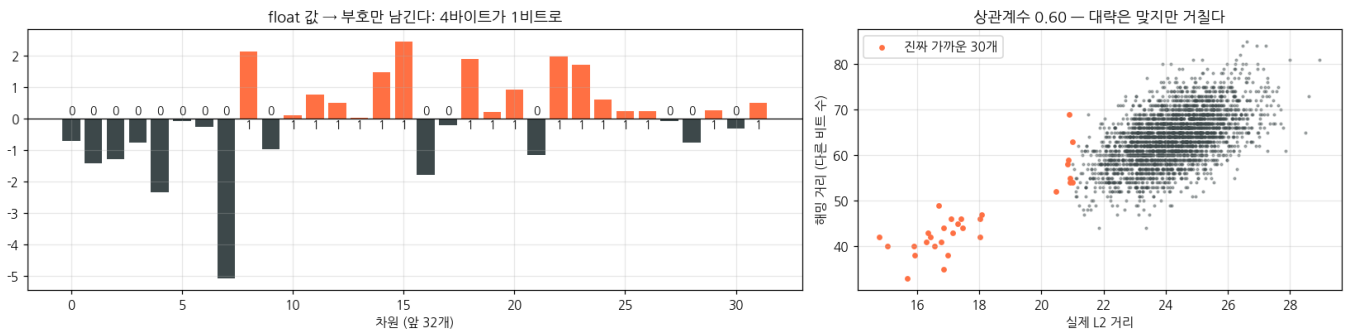
분포가 바뀌면 중심 재학습이 필요하다



1. 군집 100 개일 때 인덱스를 만들고 나머지 100 개 군집을 INSERT
- 새 벡터는 옛 중심 중 가까운 리스트로
2. 새 군집에 정답이 있는 질의 recall 0.49
-REINDEX (0.8 초) 후 1.00
3. 빈 테이블에 IVFFlat 을 먼저 만들지 말 것

pgvector 에는 PQ 가 없다 → halfvec · bit

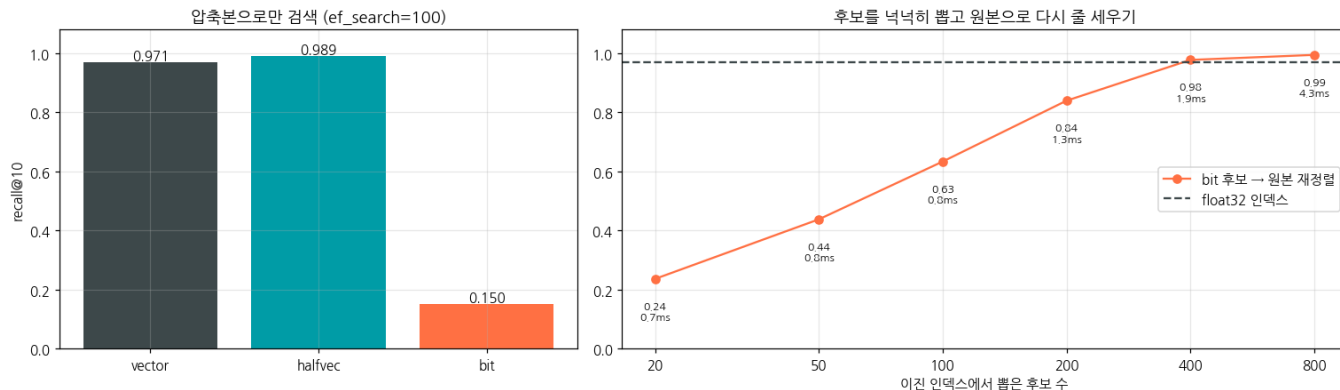
```
CREATE INDEX ON vec_clu USING hnsw ((emb::halfvec(128)) halfvec_l2_ops);
CREATE INDEX ON vec_clu USING hnsw ((binary_quantize(emb)::bit(128)) bit_hamming_ops);
```



	float32	halfvec	1 비트
인덱스	79.4MB	54.4MB	30.4MB
빌드	5.1 초	4.1 초	3.7 초

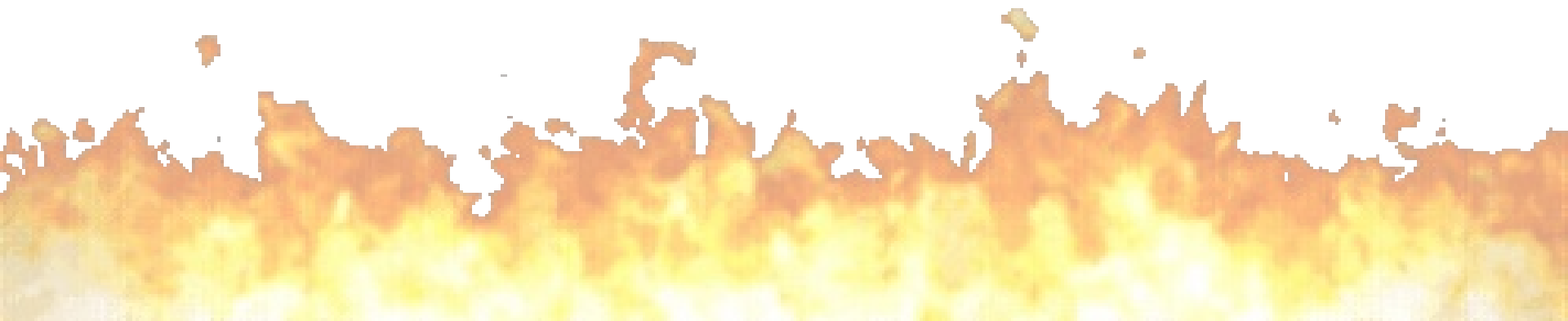
인덱스는 작게, 정확도는 원본으로

```
SELECT id FROM (SELECT id, emb FROM vec_clu  
                ORDER BY binary_quantize(emb)::bit(128) <~> binary_quantize($q) LIMIT 400) c  
ORDER BY emb <-> $q LIMIT 10;
```



halfvec 0.981 $\approx$ float32 0.985 $\cdot$ 1 비트 단독 0.149 $\rightarrow$ 후보 400 개 재정렬 0.975 (2ms)
13 번 PQ + 재정렬과 같은 원리 : 인덱스는 RAM 을 아끼고, 순위는 원본이 정한다

필터와 벡터 검색



tag < 1 (1%) + HNSW

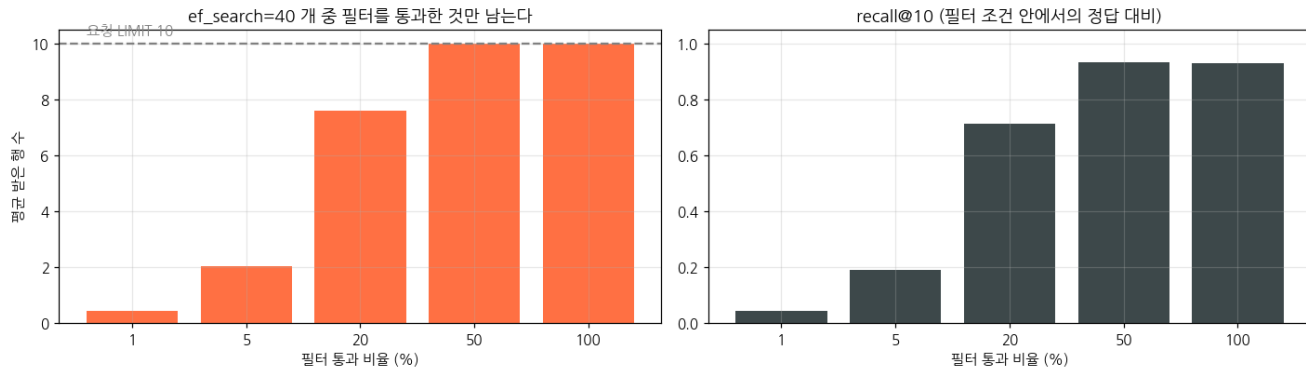
Limit (actual rows=0)

-> Index Scan using vec_clu_hnsw on vec_clu

Filter: (tag < 1)

Rows Removed by Filter: 40

-- ef_search 40 개 중 통과 0



1% 필터면 LIMIT 10 에 평균 0.4 행 — 4-1 슬라이드 'PostFilter : 최종 결과가 아예 없음' 이 이것

B-tree 가 있어도 플래너는 HNSW 를 고른다

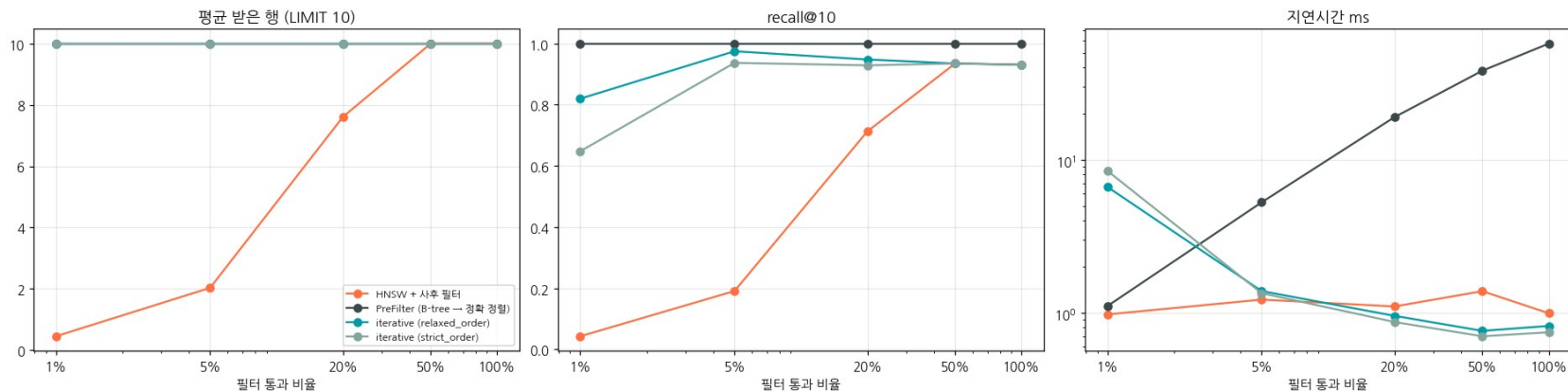
```
WITH c AS MATERIALIZED (SELECT id, emb FROM vec_clu WHERE tag < 1)
SELECT id FROM c ORDER BY emb <-> $q LIMIT 10;
```

```
CTE c -> Bitmap Index Scan on vec_clu_tag (actual rows=994)
-> Sort -> CTE Scan on c -- 994 행만 정확히 정렬
```

LIMIT 10 이 붙으면 플래너는 HNSW 스캔이 가장 싸다고 보고 필터는 나중에 건다
필터를 먼저 실행하게 가두면 언제나 정확 (recall 1.0), 비용은 통과한 행 수에 비례

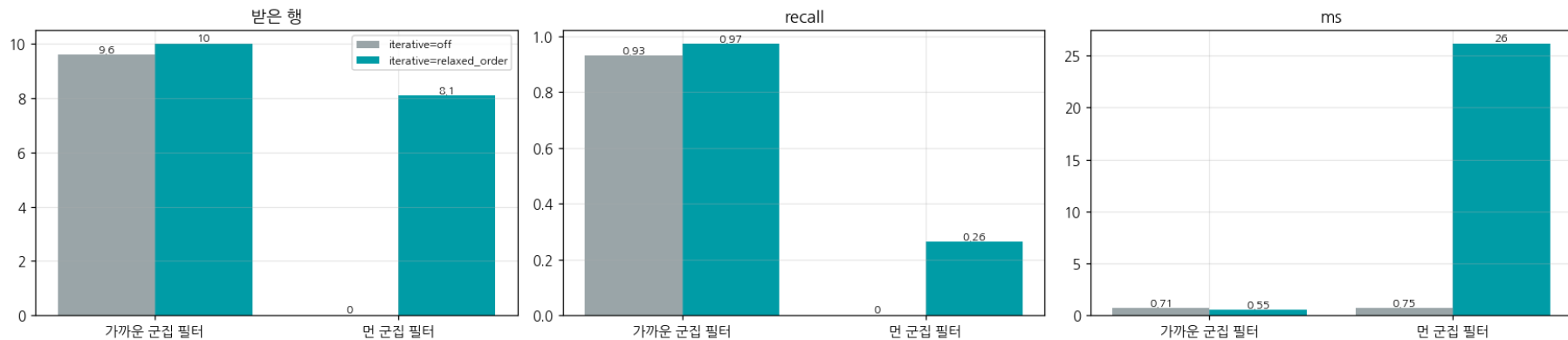
선택도에 따라 최선이 바뀐다

```
SET hnsw.iterative_scan = relaxed_order; -- 모자라면 그래프를 더 탐색 (상한 2만 행)
```



좁으면 (1%) PreFilter · 중간은 iterative scan · 넓으면 사후 필터 — 4-1 「필터링 선택 가이드」 그대로
iterative relaxed : 1% 필터에서도 10 행 · recall 0.82

선택도가 같아도 (0.5%) 방향이 다르면



1. 질의와 같은 군집으로 필터 : 사후 필터로도 9 행 · 0.8ms

2. 먼 군집으로 필터 : iterative 로도 recall 0.27 · 26ms — 탐색 상한까지 헤맨다

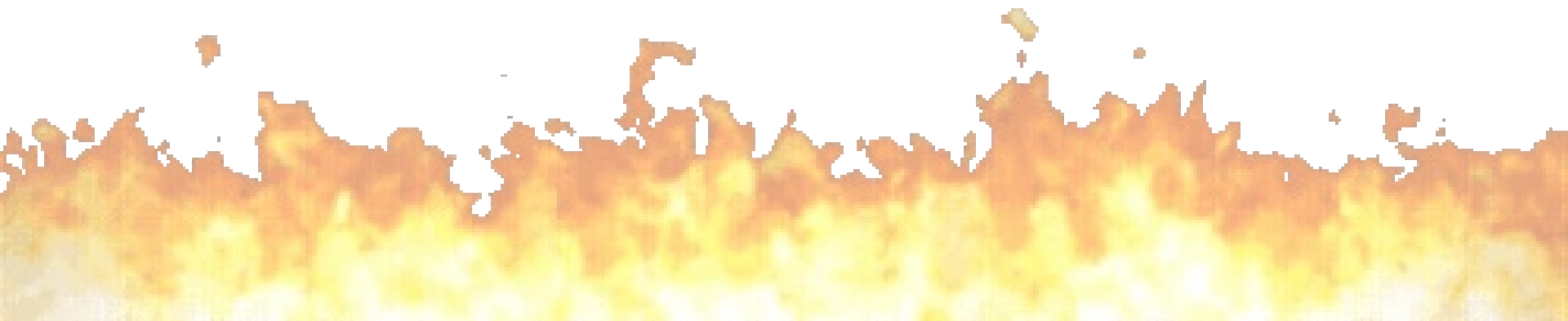
자주 쓰는 필터는 그 행만으로 그래프를 만든다

```
CREATE INDEX vec_clu_hnsw_tag7 ON vec_clu USING hnsw (emb vector_l2_ops) WHERE tag = 7;  
-- 약 1천 행, 0.9 초
```

	recall@10	받은 행	ms
전체 HNSW + 사후 필터	0.035	0.39	0.6
부분 HNSW (WHERE tag = 7)	1.000	10	0.3

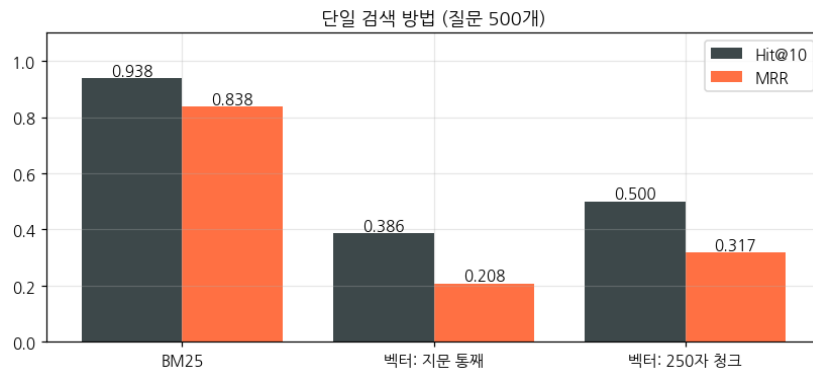
4-1 슬라이드 '부분집합 그래프는 불가능' → PG에서는 부분 인덱스로 가능
값이 적고 고정된 필터 (테넌트 · 카테고리) 에 . 값이 많으면 파티셔닝

점수 퓨전과 분포 진단



KLUE-MRC 질문 500 개, 각 상위 50 개

```
SELECT doc_id, max(sim) FROM (  
    SELECT doc_id, 1 - (embedding <=> $q) AS sim FROM doc_chunks  
    ORDER BY embedding <=> $q LIMIT 200) c          -- 청크 200 개  
GROUP BY doc_id ORDER BY 2 DESC LIMIT 50;          -- 지문별 최고 유사도
```

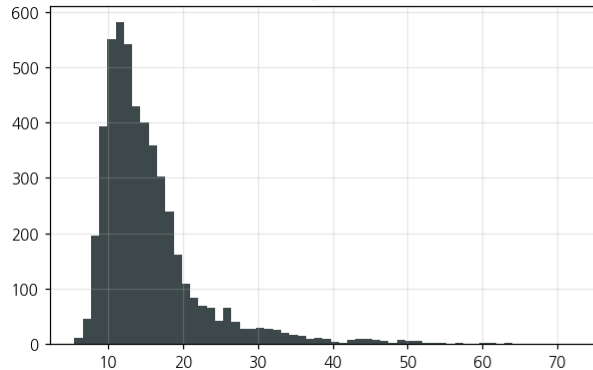


청크 임베딩으로 벡터 Hit@10 0.39 → 0.50 (4-1 의 '지문 뒤쪽은 벡터에 안 담긴다' 를 일부 해결)
그래도 BM25 0.94 — 질문이 지문의 고유명사를 그대로 쓰는 데이터

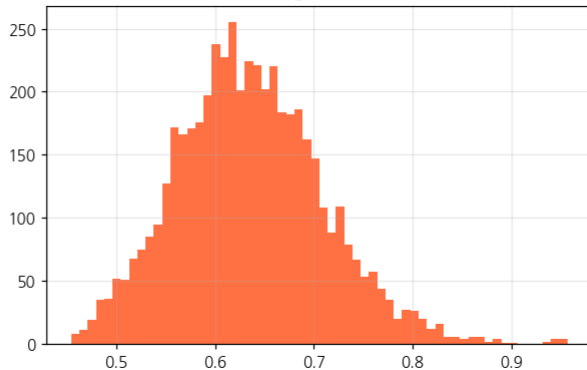
점수 척도가 다르다

그냥 더하면 안 되는 이유

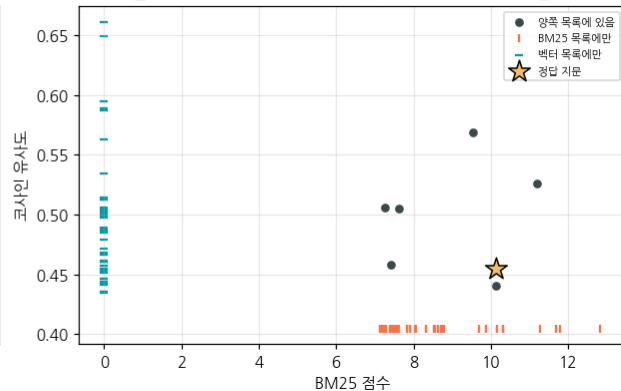
BM25 점수 (상위 10): 5 ~ 72



코사인 유사도 (상위 10): 0.45 ~ 0.96



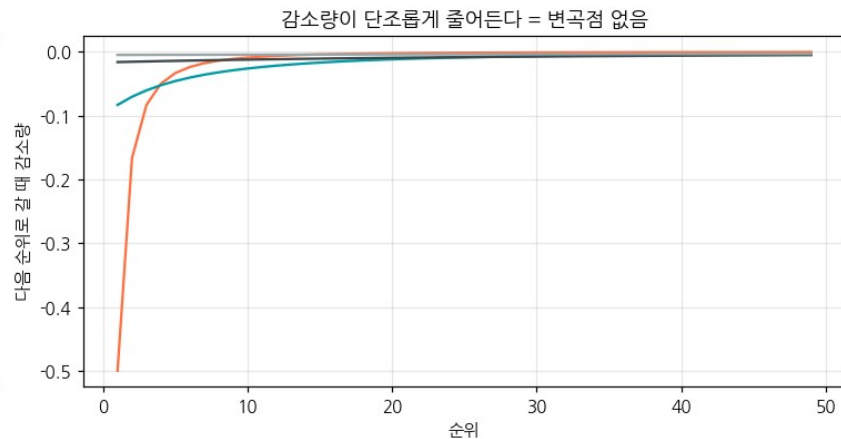
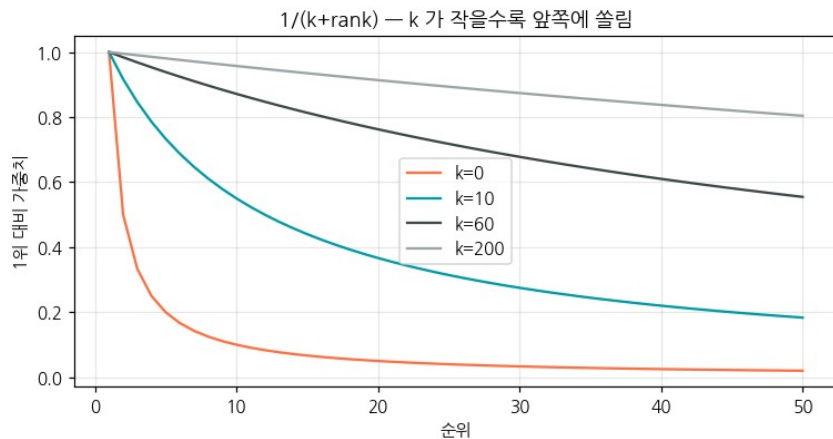
질문 하나의 후보 — 한쪽에만 있으면 다른 쪽 점수가 없다



BM25 5~72 · 코사인 0.45~0.96 → 더하면 BM25 가 다 정한다
한쪽 목록에만 있는 문서는 다른 쪽 점수가 아예 없다

강의의 '변곡점 조절' 과 다르다

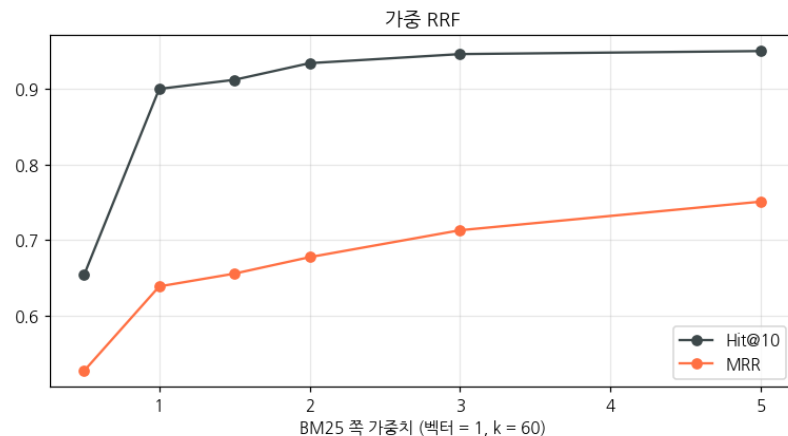
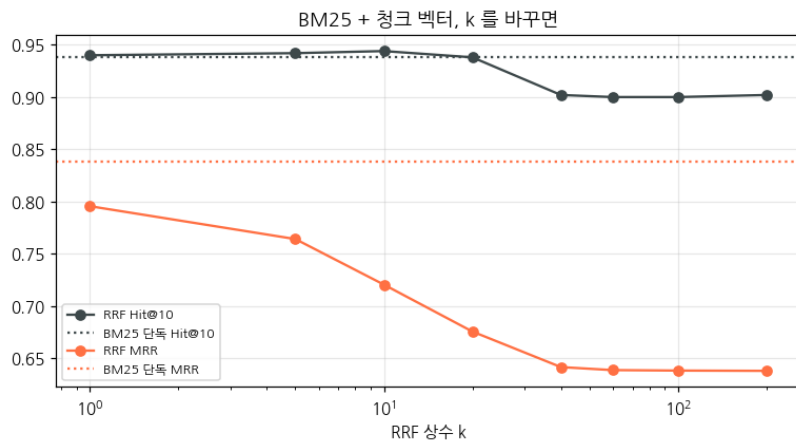
$$\text{RRF}(d) = \sum 1 / (k + \text{rank})$$



쌍곡선이라 변곡점은 없다. k 는 앞 순위 쏠림 정도 : 10 위 / 1 위 가중치 = k=0 에서 0.10, k=60 에서 0.87

RRF : k 와 가중치

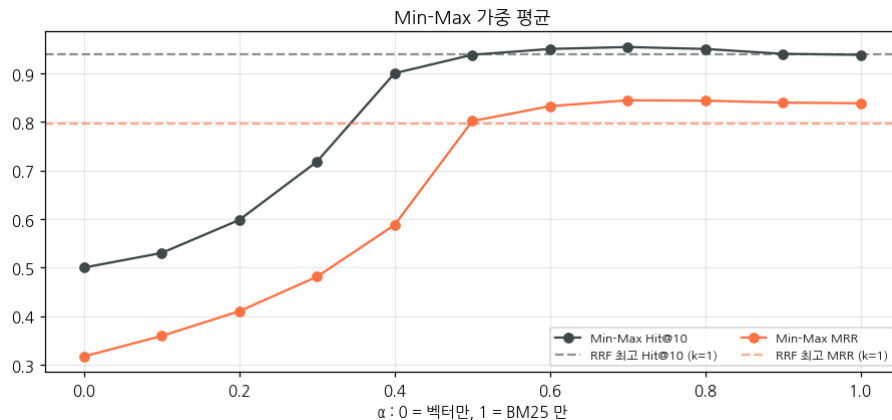
흔한 $k=60$ 은 약한 목록과 동등하게 섞는다



	BM25 단독	RRF k=60	RRF k=1	가중 RRF (BM25×5)
Hit@10 / MRR	0.938 / 0.838	0.900 / 0.639	0.940 / 0.796	0.950 / 0.751

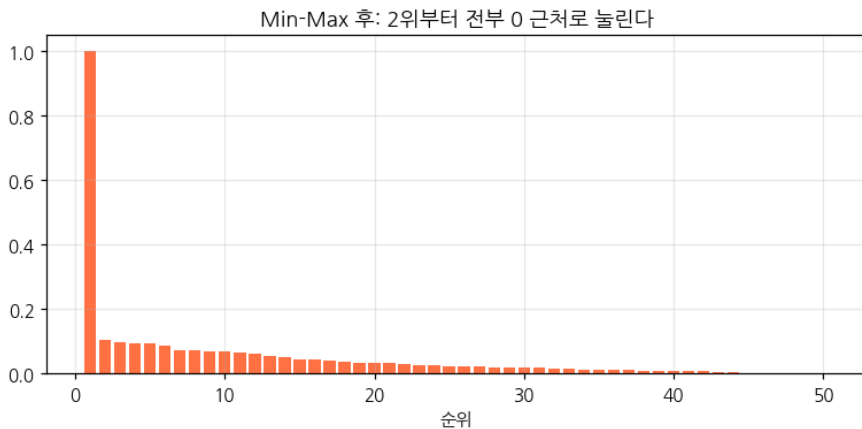
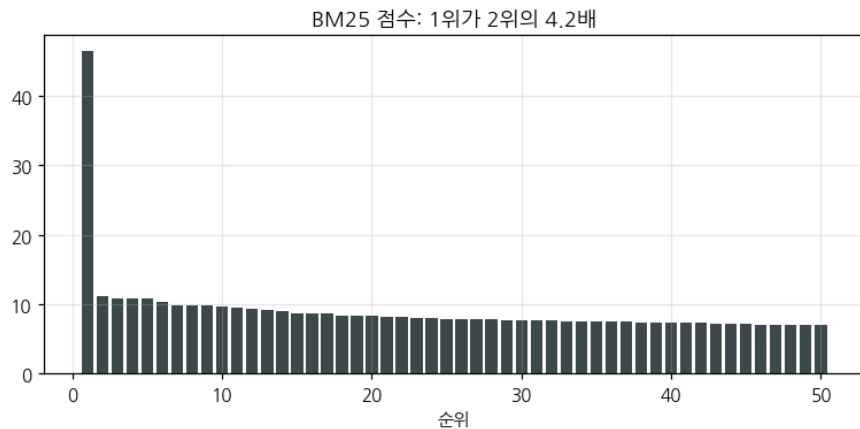
점수를 0~1 로 퍼서 가중 평균

$mm(s) = (s - \min) / (\max - \min)$ # 질문마다, 각 목록 안에서
 $fused = \alpha \cdot mm(BM25) + (1 - \alpha) \cdot mm(cos)$ # 한쪽에만 있으면 다른 쪽 0



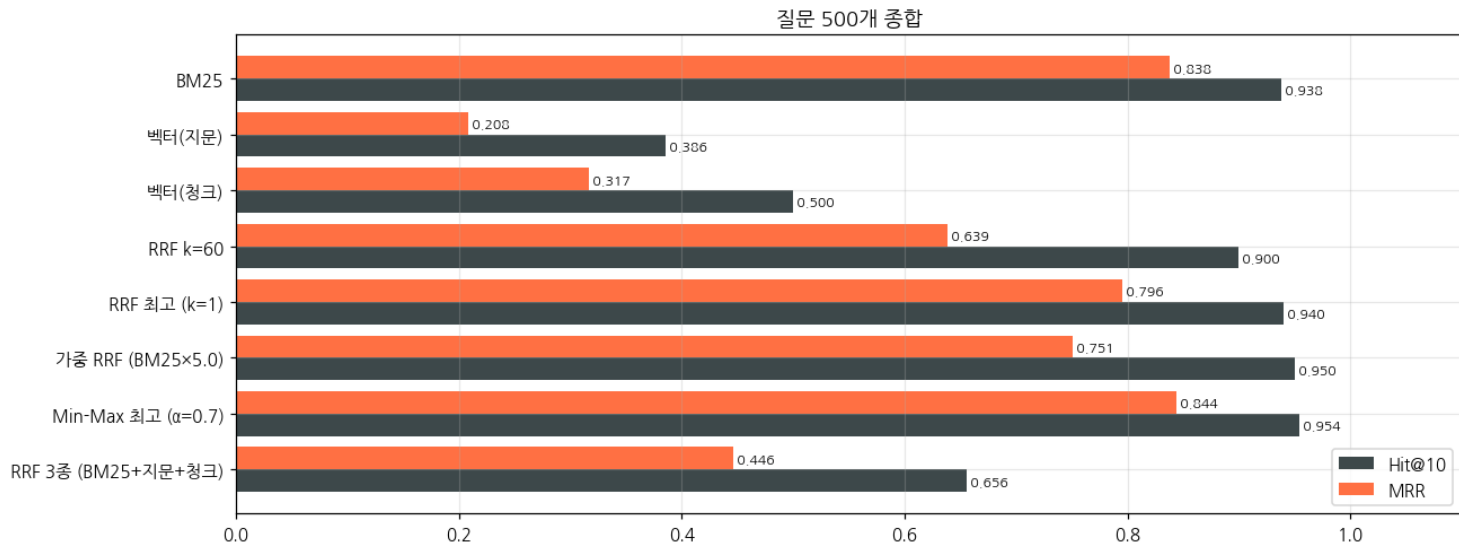
$\alpha = 0.7 : \text{Hit@10 } 0.954 \cdot \text{MRR } 0.844$ — 이 실습에서 BM25 단독을 넘은 유일한 조합 (점수 크기 = 확신을 살린다)

극단값 하나가 스케일을 정한다



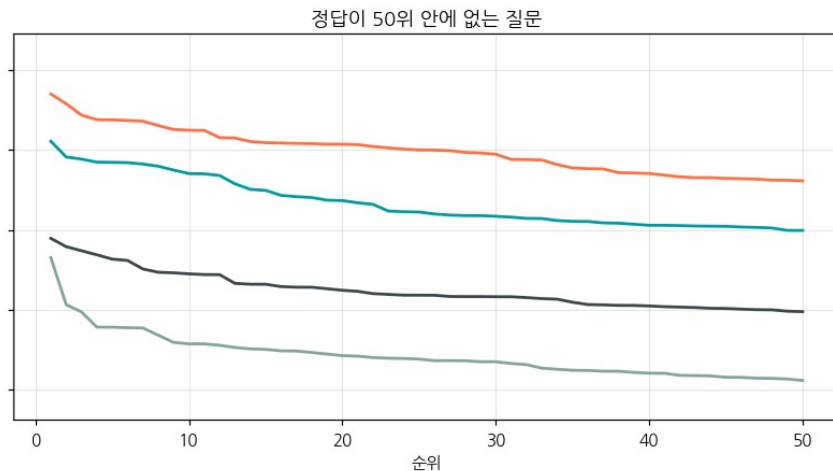
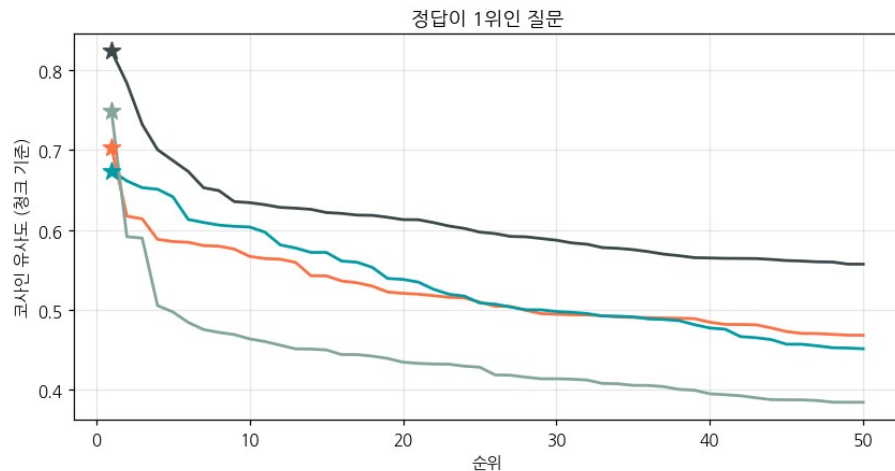
BM25 1 위가 압도적인 질문 : 정규화 후 2 위부터 0 근처로 눌러 벡터 점수가 조금만 높아도 뒤집힌다
 α 는 데이터마다 다시 찾는다 — 평가셋을 나눠 고르고 확인해야 과적합을 피한다

질문 500 개 · Hit@10 / MRR



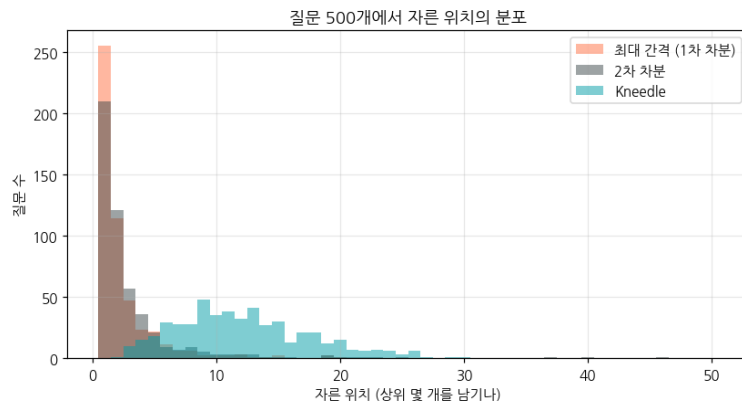
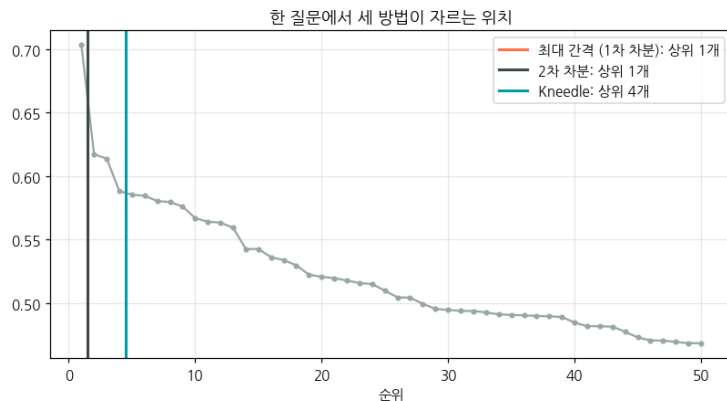
약한 목록을 섞을수록 손해 (RRF 3 종 0.656) · 점수 분포가 믿을 만하면 Min-Max, 제멋대로면 RRF

맞힌 질문 vs 틀린 질문



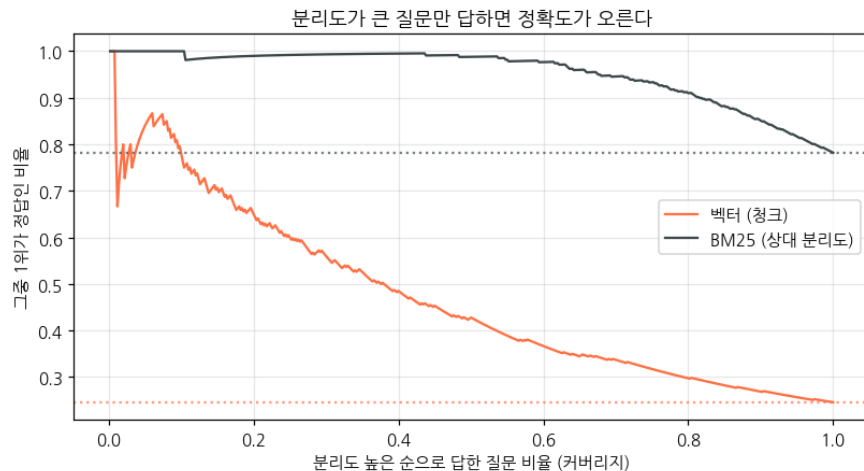
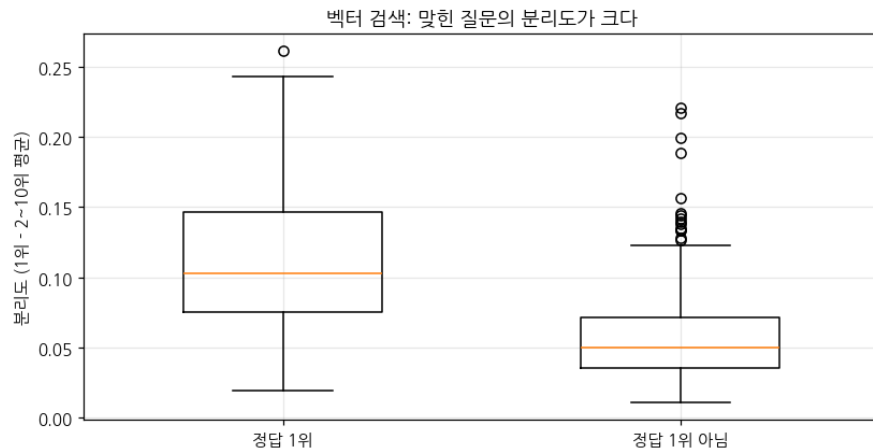
맞힌 질문은 1위가 튀어나온 곡선, 틀린 질문은 밋밋하게 내려오는 곡선 — 어디서 자를지 알 수 없는 '나쁜 분포'

2 차 차분 · 최대 간격 · Kneedle



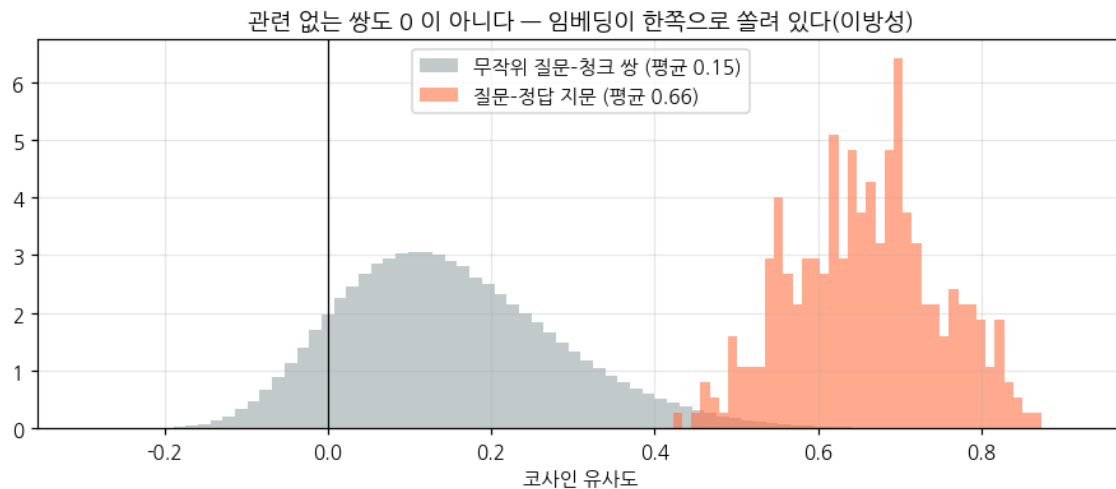
방법	평균 남긴 개수	정답이 남은 비율
최대 간격 (1 차 차분)	2.4	0.33
2 차 차분	3.1	0.35
Kneedle	12.2	0.52

1 위 점수 - 2~10 위 평균



분리도 상위 20% 만 답하면 벡터 1 위 정확도 0.25 → 약 0.7. BM25 도 같은 경향
낮으면 '모르겠다' · BM25 로 넘기기 · 질의 재작성 — 정답셋 없이 운영 중에 켤 수 있다

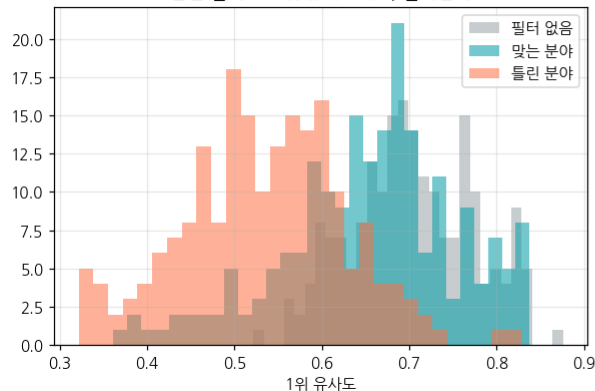
관련 없는 쌍도 0 이 아니다



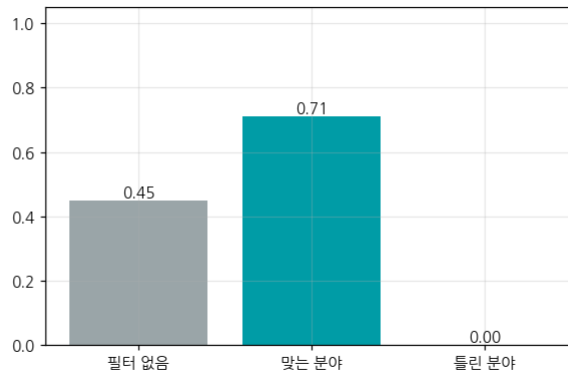
무작위 질문-청크 쌍 평균 0.15 (이방성) · 정답 쌍 평균 0.66, 두 분포가 겹친다
' 유사도 0.6 이상만 ' 같은 고정 임계값은 모델마다 의미가 다르다 → 곡선 모양을 본다

틀린 필터는 1 위 유사도로 드러난다

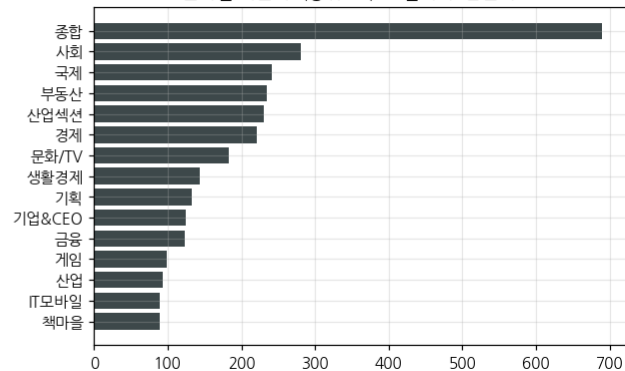
틀린 필터 → 1위 유사도가 확 떨어진다



Hit@10

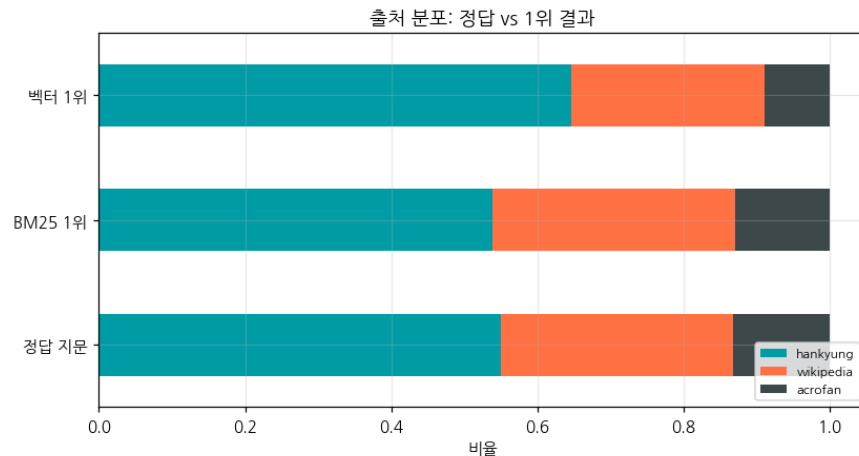
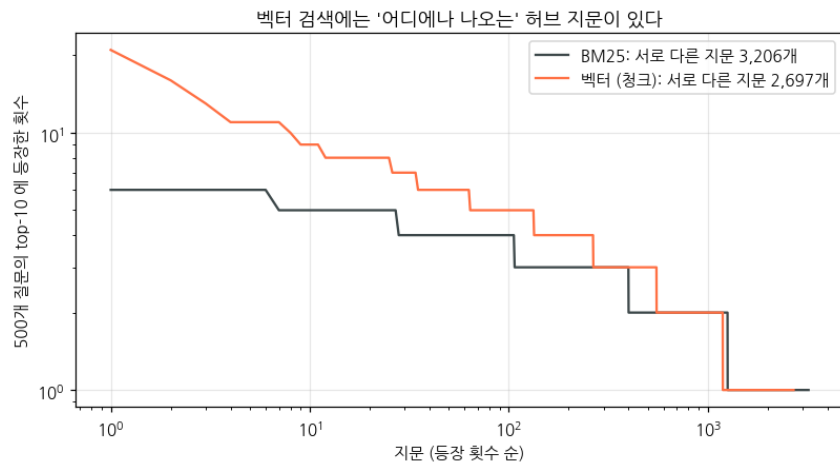


분야별 지문 수 (상위 15) — 필터 후 남는 후보



뉴스 질문 200 개 : 맞는 분야 필터 Hit@10 0.45 → 0.71 (후보가 수백 개로 준다)
틀린 분야 필터는 1 위 유사도부터 급락 — 강의 슬라이드 25 ' 유사도가 낮으면 시맨틱 불일치 '

특정 지문으로 결과가 쏠리는가



벡터 검색 : 지문 하나가 질문 21 개의 top-10 에 등장 (BM25 는 최대 6) — 고차원 임베딩의 허브 현상
 벡터 1 위의 한국경제 비율 65% vs 정답 55% — 출처 쏠림은 코퍼스 · 청킹 점검 신호

강의 슬라이드	실습 결과
4 · HNSW 는 metric-agnostic	인덱스는 빌드한 척도 하나에 묶인다 (21)
5 · 이웃 = 가장 가까운 M 개	실제는 다양성 휴리스틱 : 0.99 vs 0.72 (11)
14 · 512 차원을 8 차원으로 압축	8 차원 벡터가 아니라 8 바이트 코드 (13)
16 · 같은 속도에서 HNSW 가 최고	군집이 뚜렷하면 IVFFlat 이 낫다 (22)
16 · 범주가 뚜렷하면 IVF	맞다 . 범주가 없으면 IVF 는 무너진다 (22)
20 · RRF 의 k 로 변곡점 조절	변곡점은 없다 . k 는 쓸림 정도 (30)
24 · 2 차 미분으로 변곡점	잡음에 민감 . 간격 · Kneedle · 분리도를 함께 (31)
4-1 · 부분집합 그래프는 불가능	PG 부분 인덱스로 가능 : recall 1.0 (24)

```
docker compose down      # 컨테이너만 정지 ( 데이터 유지 )  
docker compose down -v  # 데이터까지 삭제
```

1. recall 이 모든 설정에서 같다
 - 다른 노트북이 남긴 벡터 인덱스를 플래너가 쓰는 경우 → 첫 셀부터 다시
2. 30 번 'KLUE 준비가 끝나야 한다'
 - 00 번 진행 확인 셀 · prepare.log 확인
3. 과제
 - 21 · 22 번을 KLUE 청크 임베딩 (384 차원) 으로 반복해 보기
 - 31 번의 분리도가 낮은 질문만 BM25 로 넘기는 라우터 → Hit@10 비교